

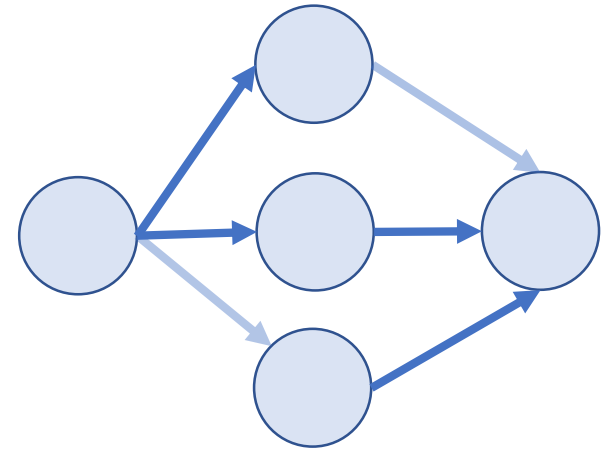
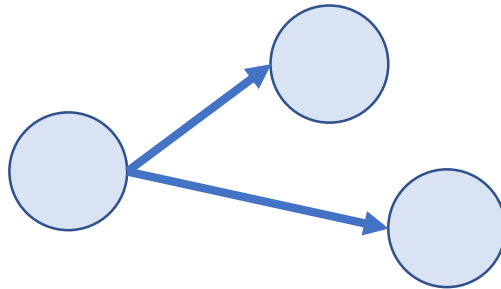
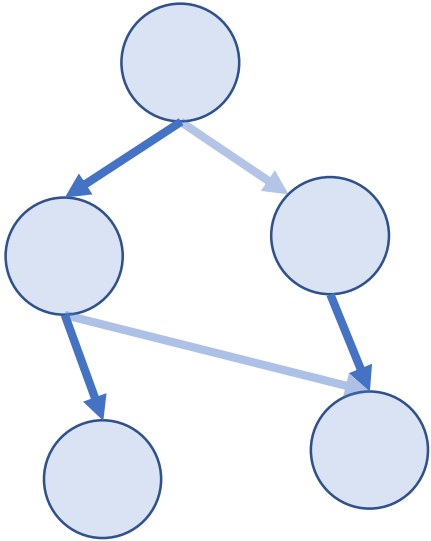
# Simulation, Data Analysis, and Robustification of Uncertain Networks Pertinent to Targeting

*Alexander Michos: FPS/KAU*

*APL Pathways Intern {2025, Graph Analytics}*

*Mentor: Taylor Ordines*

*Taylor.Ordines@jhuapl.edu*



Classification: None / Unclassified

# Why Study Uncertainty?

**Probability distributions** allow us to model factors that may be **beyond our control**:

- Ex: Communication channels have **capacities** and **latencies** that are **randomly** evolving according to a Latency Distribution.

# Why Study Uncertainty?

**Probability distributions** allow us to model factors that may be **beyond our control**:

- Ex: Communication channels have **capacities** and **latencies** that are **randomly** evolving according to a Latency Distribution.

**Noisiness / uncertainty** provides great **sensitivity and importance analysis** for network parameters:

- Ex: Which edge/node would **upgrading** provide the greatest benefit to the system?

# Why Study Uncertainty?

**Probability distributions** allow us to model factors that may be **beyond our control**:

- Ex: Communication channels have **capacities** and **latencies** that are **randomly** evolving according to a Latency Distribution.

**Noisiness / uncertainty** provides great **sensitivity and importance analysis** for network parameters:

- Ex: Which edge/node would **upgrading** provide the greatest benefit to the system?

**Uncertainty** can also be used to study the **reliability/robustness** of the network to unforeseen disruptions:

- Ex: **Adversaries** may target components that have a disproportionate **influence** on end-to-end performance.

# Why Study Uncertainty?

**Probability distributions** allow us to model factors that may be **beyond our control**:

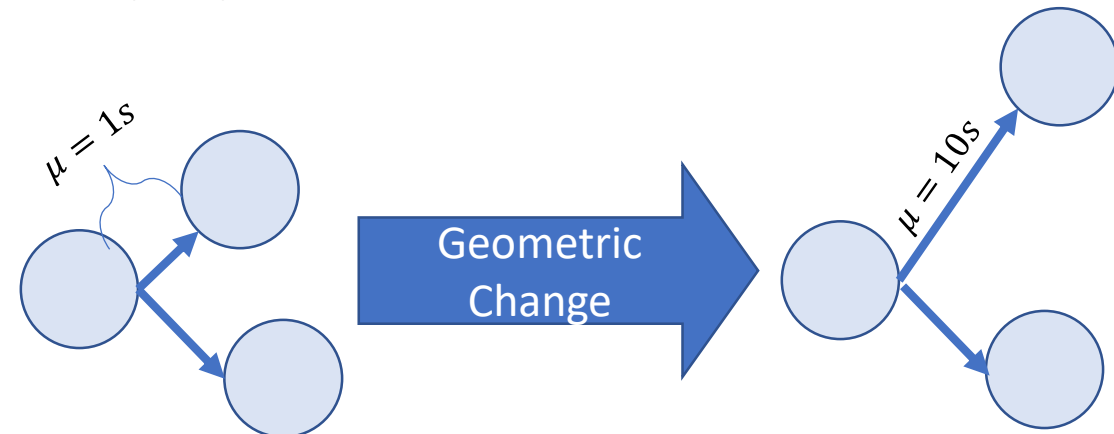
- Ex: Communication channels have **capacities** and **latencies** that are **randomly** evolving, (Latency Distribution)

**Noisiness / uncertainty** provides great **sensitivity and importance analysis** for network parameters:

- Ex: Which edge/node would **upgrading** provide the greatest benefit to the system?

**Uncertainty** can also be used to study the **reliability/robustness** of the network to unforeseen disruptions:

- Ex: **Adversaries** may target components that have a disproportionate **influence** on end-to-end performance.



# The Philosophy:

At a high level, uncertain networks were studied with the following strategy:

- Program a simulation which tests networks of a certain type.
- Run **Monte-Carlo trials** over different configurations / parameters.
  - Evaluate network performance using **metrics** & statistics.
- Identify trends among the feature – performance data pairs.

# The Philosophy:

At a high level, uncertain networks were studied with the following strategy:

- Program a simulation which tests networks of a certain type.
- Run **Monte-Carlo trials** over different configurations / parameters.
  - Evaluate network performance using **metrics** & statistics.
- Identify trends among the feature – performance data pairs.

## Baseline Network:

Connectivity,  
Latency  
Distributions, etc.

*"This is what we  
have / need."*

# The Philosophy:

At a high level, uncertain networks were studied with the following strategy:

- Program a simulation which tests networks of a certain type.
- Run **Monte-Carlo trials** over different configurations / parameters.
  - Evaluate network performance using **metrics** & statistics.
- Identify trends among the feature – performance data pairs.

## Baseline Network:

Connectivity,  
Latency  
Distributions, etc.

## Simulation:

End-to-End Network  
Performance  
for Random Inputs /  
Perturbations

*"This is what we  
have / need."*





# The Philosophy:

At a high level, uncertain networks were studied with the following strategy:

- Program a simulation which tests networks of a certain type.
- Run **Monte-Carlo trials** over different configurations / parameters.
  - Evaluate network performance using **metrics** & statistics.
- Identify trends among the feature – performance data pairs.

## Baseline Network:

Connectivity,  
Latency  
Distributions, etc.

*"This is what we  
have / need."*

## Simulation:

End-to-End Network  
Performance  
for Random Inputs /  
Perturbations

## Evaluation:

Reliability, Efficacy,  
Latency,  
Performance  
Requirements, etc.

*"How well does it  
work?"*



# The Philosophy:

At a high level, uncertain networks were studied with the following strategy:

- Program a simulation which tests networks of a certain type.
- Run **Monte-Carlo trials** over different configurations / parameters.
  - Evaluate network performance using **metrics** & statistics.
- Identify trends among the feature – performance data pairs.

**Baseline Network:**  
Connectivity,  
Latency  
Distributions, etc.

*"This is what we  
have / need."*

**Simulation:**  
End-to-End Network  
Performance  
for Random Inputs /  
Perturbations



**Evaluation:**  
Reliability, Efficacy,  
Latency,  
Performance  
Requirements, etc.

*"How well does it  
work?"*

**Data Analysis:**  
Sensitivity Analysis,  
Recommendations,  
Regression

*"What should we do  
about it?"*

# Types of Networks: Shortest-Latency Paths

## Stochastic-Latency Networks:

- Edges have a **random** latency (i.e. time to traverse) sampled from its **latency distribution**, together with a non-zero **probability of failure**.

# Types of Networks: Shortest-Latency Paths

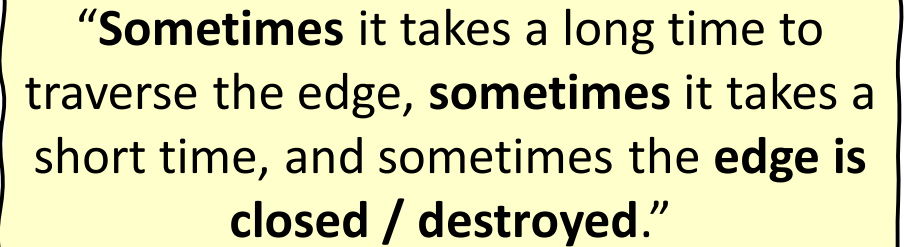
## Stochastic-Latency Networks:

- Edges have a **random** latency (i.e. time to traverse) sampled from its **latency distribution**, together with a non-zero **probability of failure**.
  - Ex: Packets on a comms network must find the **shortest path** between nodes given random delays and faulty links.

# Types of Networks: Shortest-Latency Paths

## Stochastic-Latency Networks:

- Edges have a **random** latency (i.e. time to traverse) sampled from its **latency distribution**, together with a non-zero **probability of failure**.
  - Ex: Packets on a comms network must find the **shortest path** between nodes given random delays and faulty links.



**“Sometimes** it takes a long time to traverse the edge, **sometimes** it takes a short time, and sometimes the **edge is closed / destroyed.**”

# Types of Networks: Shortest-Latency Paths

## Stochastic-Latency Networks:

- Edges have a **random** latency (i.e. time to traverse) sampled from its **latency distribution**, together with a non-zero **probability of failure**.
  - Ex: Packets on a comms network must find the **shortest path** between nodes given random delays and faulty links.

“**Sometimes** it takes a long time to traverse the edge, **sometimes** it takes a short time, and sometimes the **edge is closed / destroyed.**”

How **far apart** (probabilistically) are two nodes, and where are the **bottlenecks** preventing **faster** or more **reliable** connection?

# Types of Networks: Shortest-Latency Paths

## Stochastic-Latency Networks:

- Edges have a **random** latency (i.e. time to traverse) sampled from its **latency distribution**, together with a non-zero **probability of failure**.
  - Ex: Packets on a comms network must find the **shortest path** between nodes given random delays and faulty links.

“**Sometimes** it takes a long time to traverse the edge, **sometimes** it takes a short time, and sometimes the **edge is closed / destroyed.**”

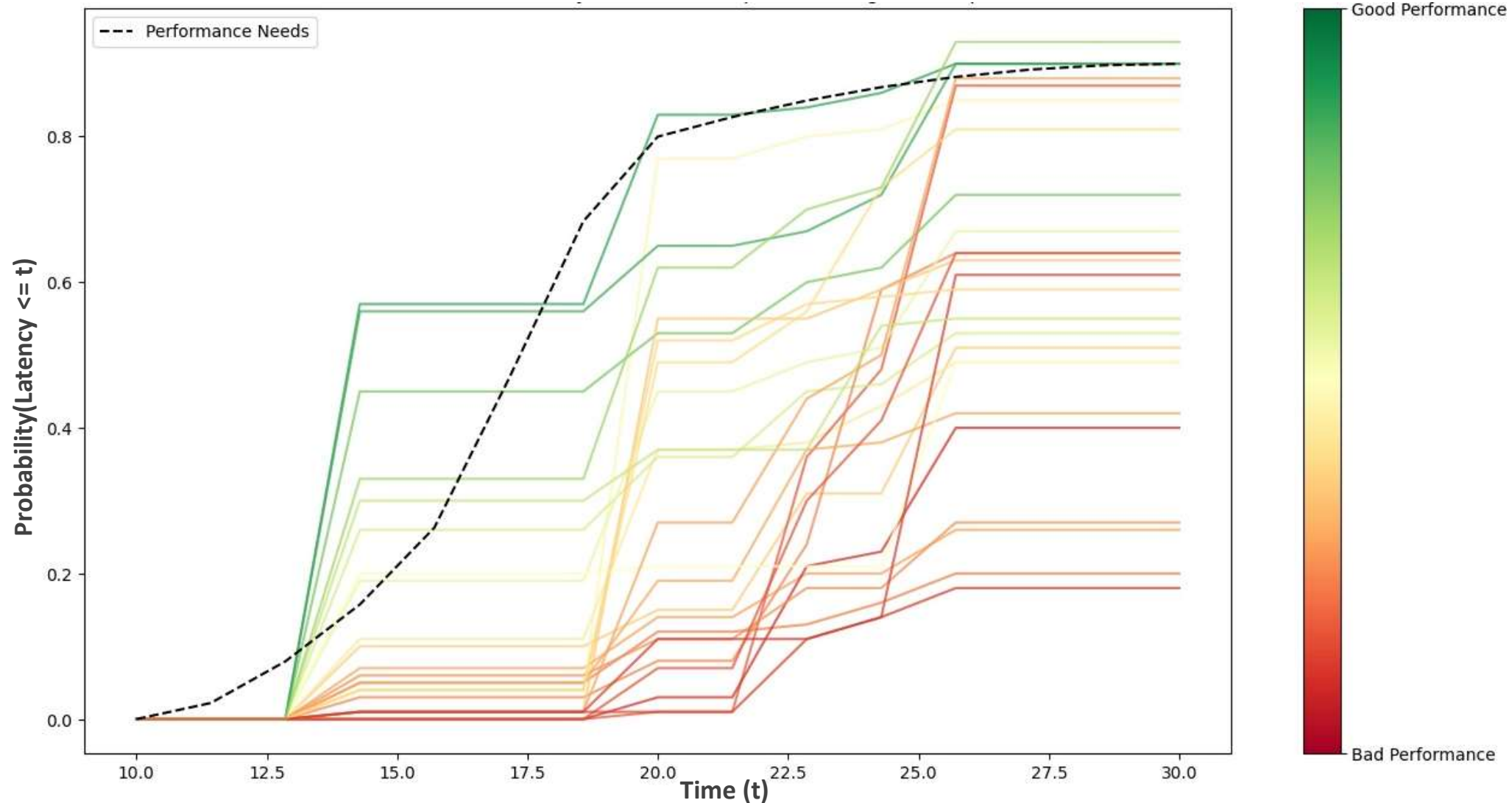
How **far apart** (probabilistically) are two nodes, and where are the **bottlenecks** preventing **faster** or more **reliable** connection?

- Formulated mathematically as a **CDF**:  $\text{Prob}(\text{Latency}[\text{Shortest Path}] \leq x)$ ;
  - If the **target vulnerability window** is randomly distributed, that's  $P(\text{Latency}[\text{Shortest Path}] - T_{\text{Vulnerable}} \leq 0)$



# Note on CDF's:

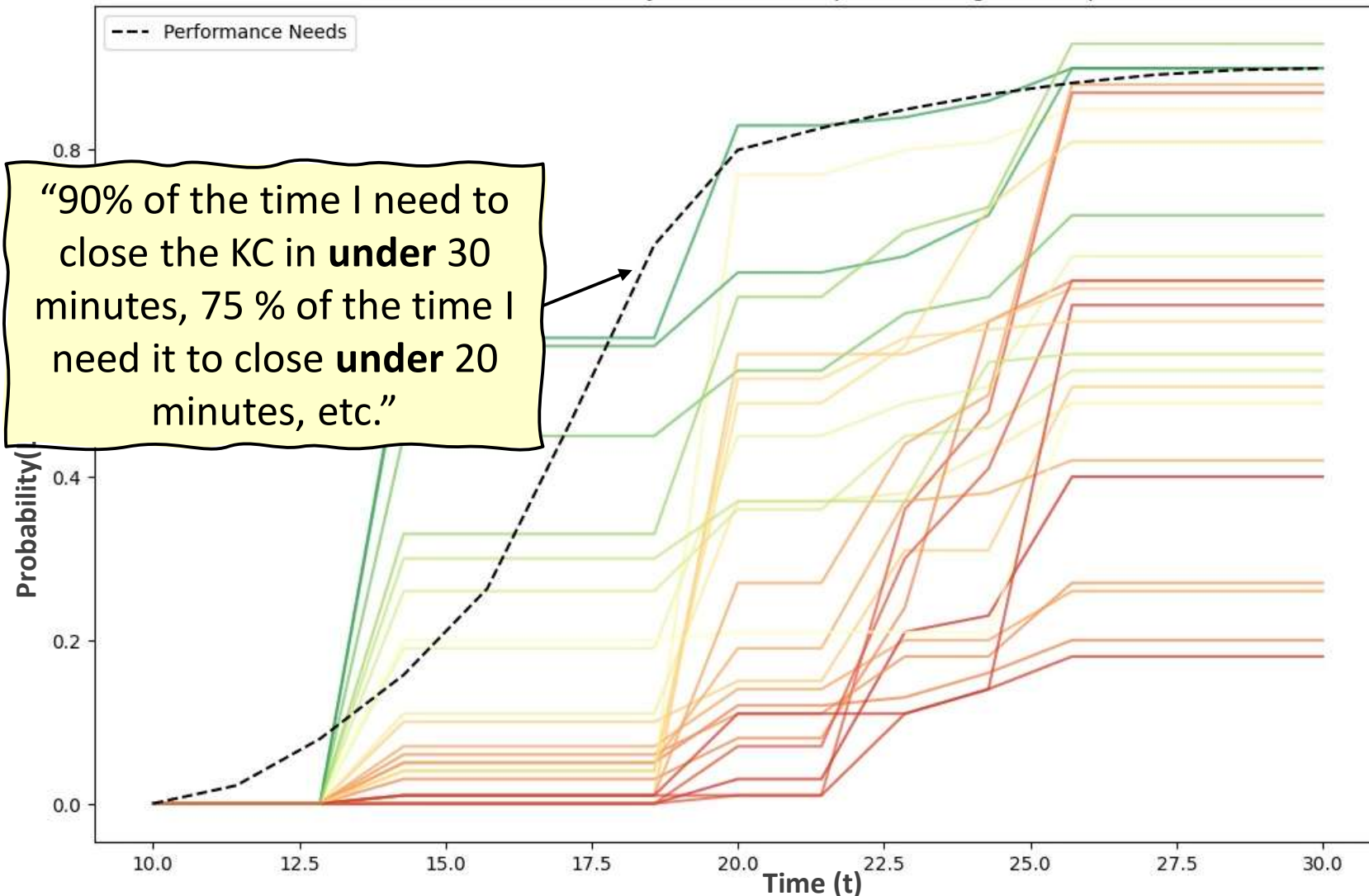
- CDF's demonstrate the “**full**” performance of a system: the average, the extremes, and everything in between, providing a **key tool** for probabilistic importance analysis.





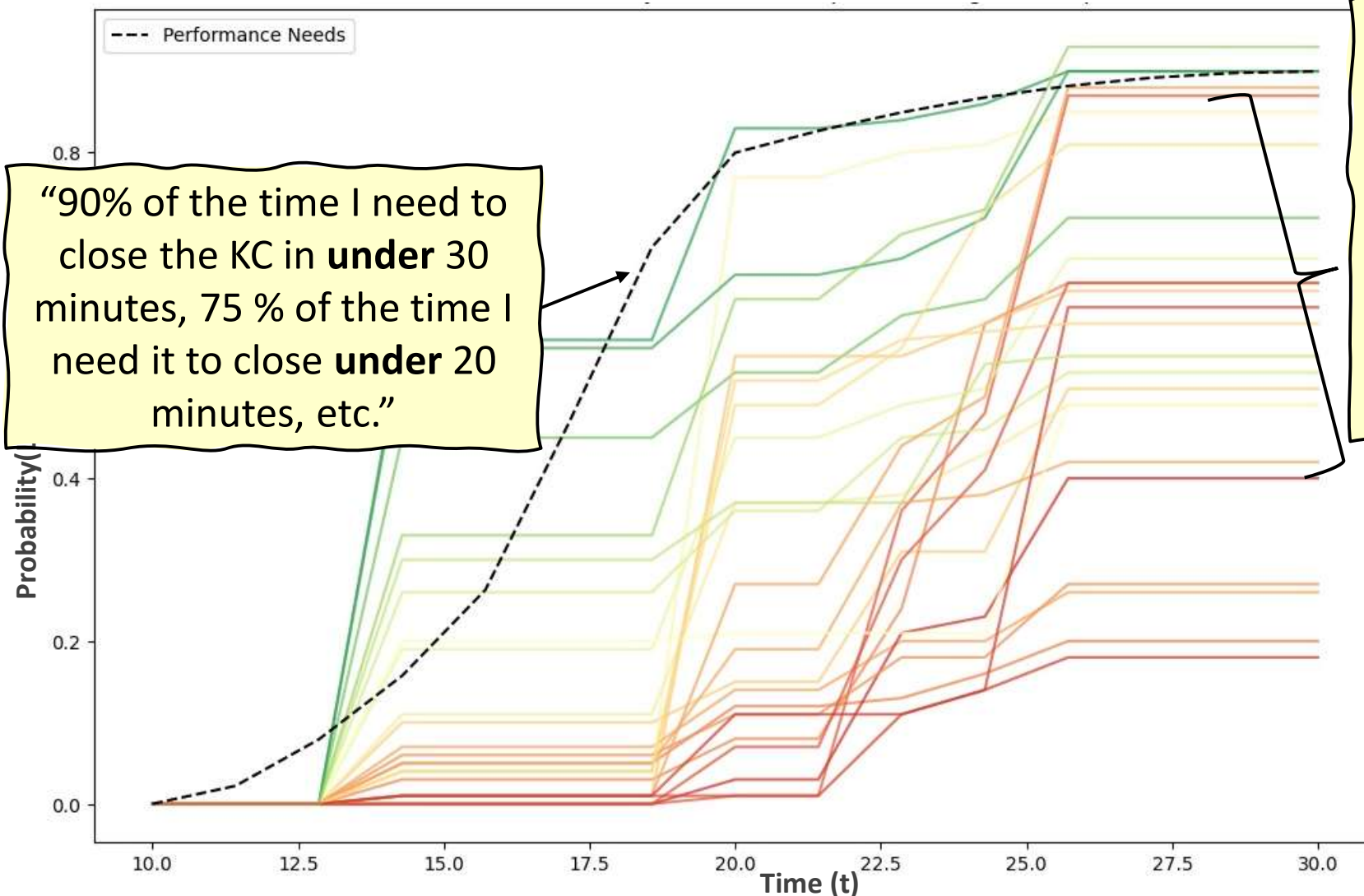
# Note on CDF's:

- CDF's demonstrate the “**full**” performance of a system: the average, the extremes, and everything in between, providing a **key tool** for probabilistic importance analysis.



# Note on CDF's:

- CDF's demonstrate the “**full**” performance of a system: the average, the extremes, and everything in between, providing a **key tool** for probabilistic importance analysis.

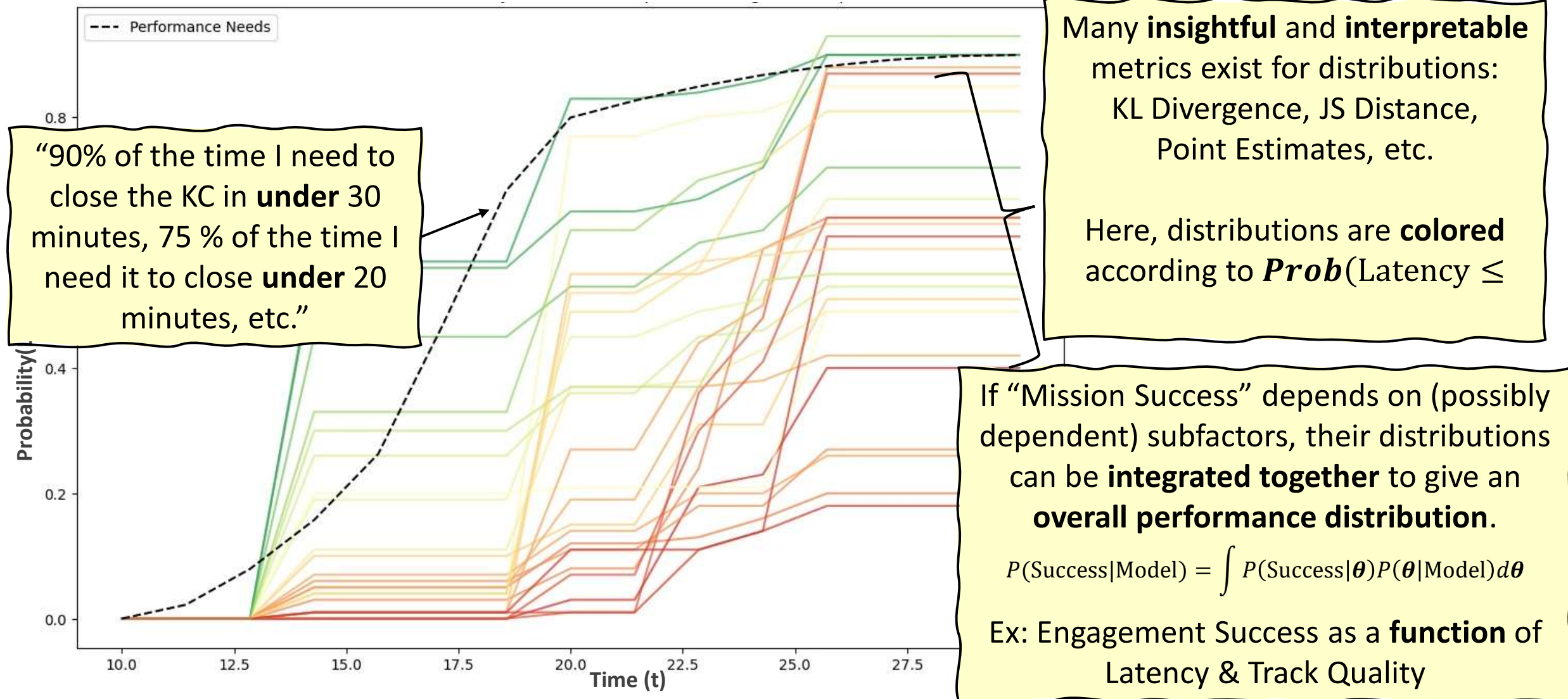


Many **insightful** and **interpretable** metrics exist for distributions: KL Divergence, JS Distance, Point Estimates, etc.

Here, distributions are **colored** according to ***Prob***(Latency  $\leq$

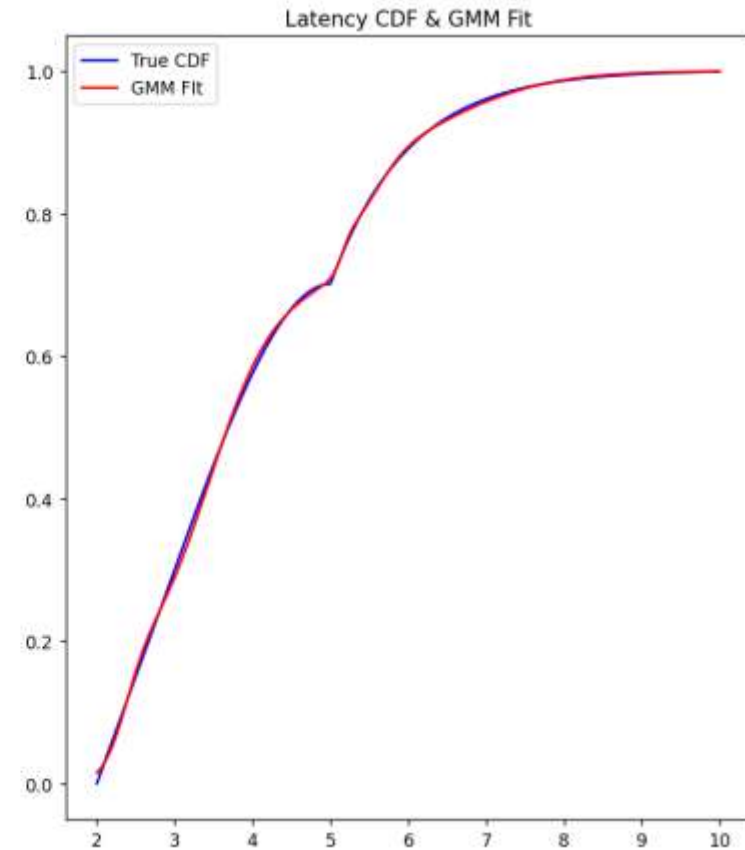
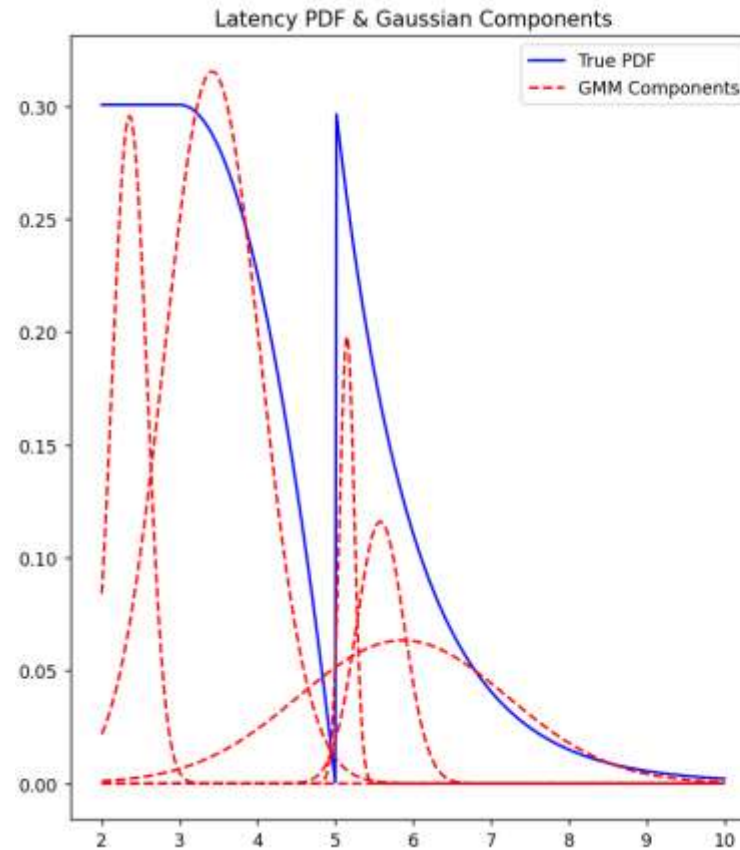
# Note on CDF's:

- CDF's demonstrate the “**full**” performance of a system: the average, the extremes, and everything in between, providing a **key tool** for probabilistic importance analysis.



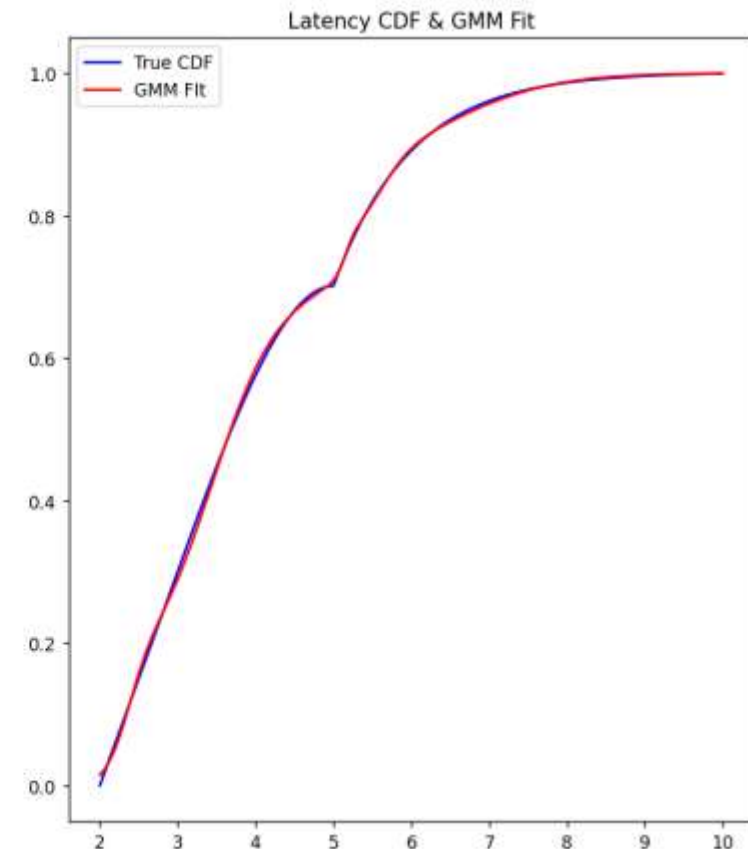
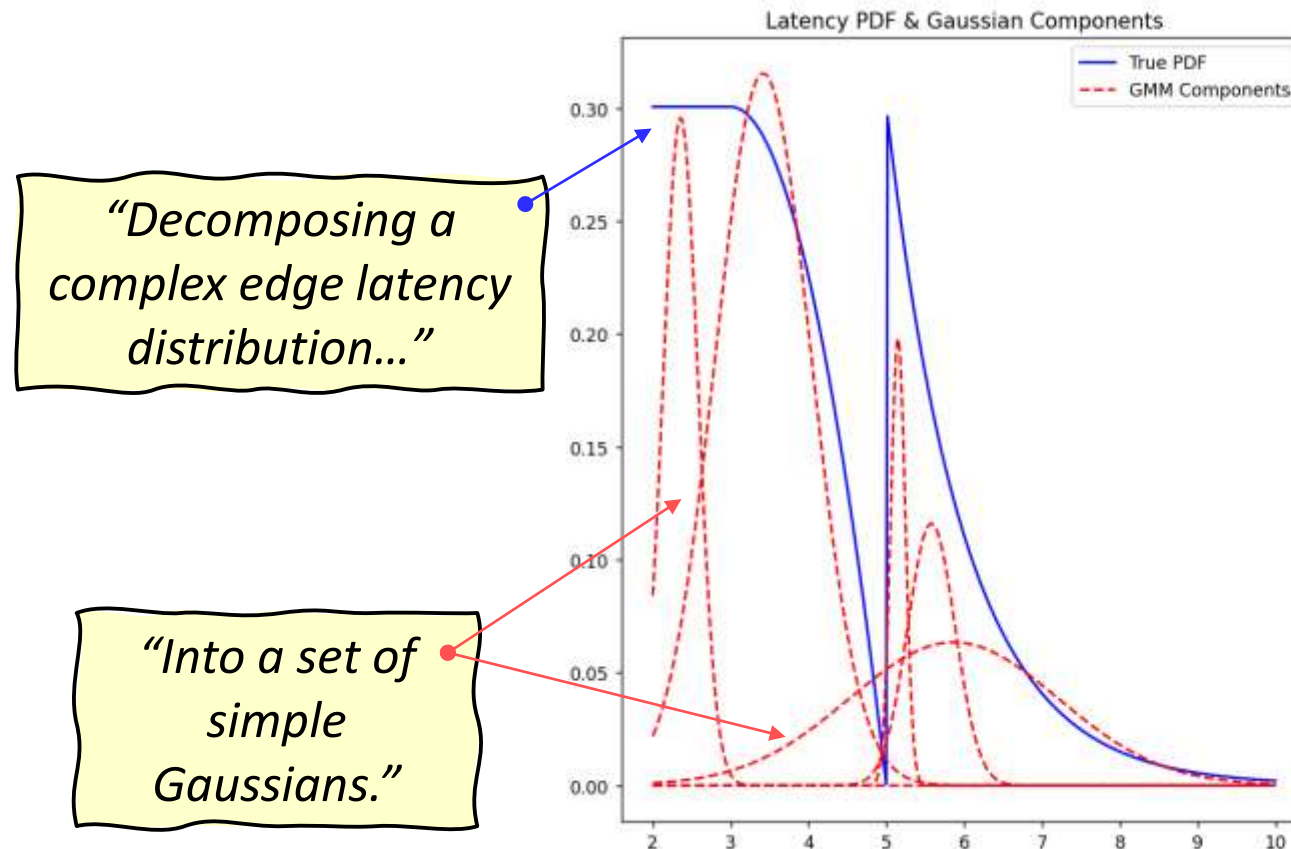
# SIMULATION: Shortest-Latency Paths

- Developed a **Gaussian Mixture Model** implementation to manipulate arbitrary edge **latency distributions**.
  - I.e. Gaussians are easy to fit (**E-M Algorithm**), sample from, integrate, and convolve:



# SIMULATION: Shortest-Latency Paths

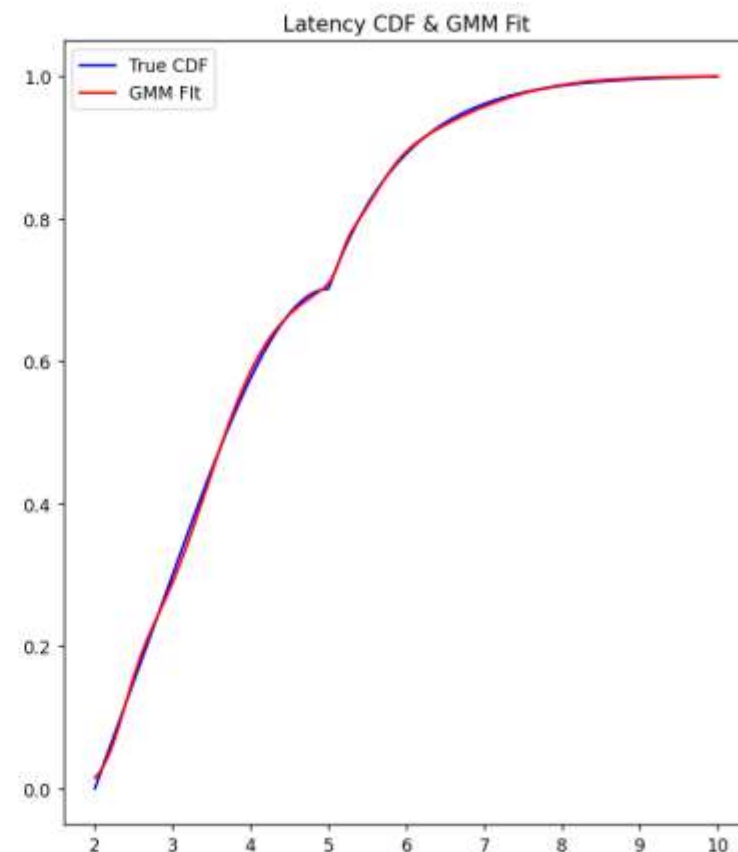
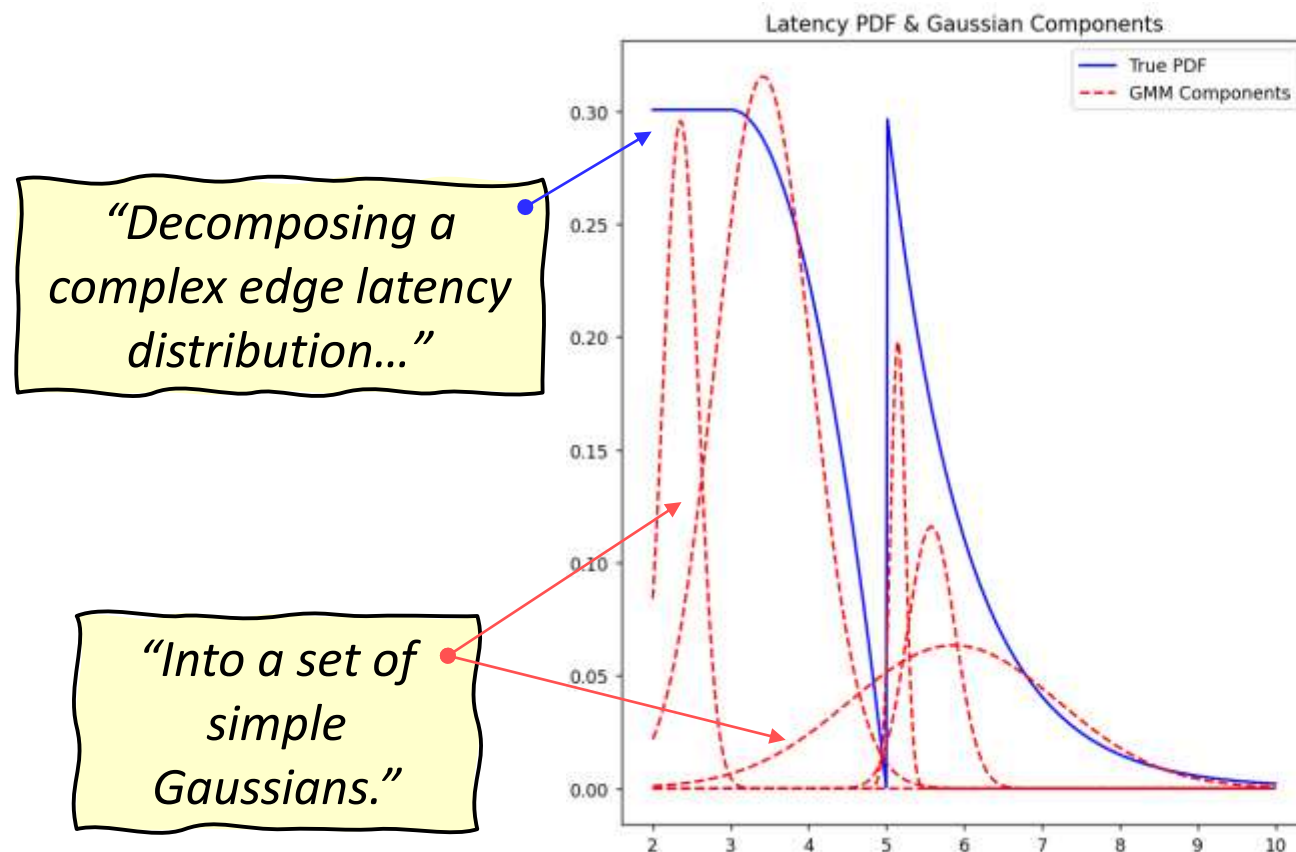
- Developed a **Gaussian Mixture Model** implementation to manipulate arbitrary edge **latency distributions**.
  - I.e. Gaussians are easy to fit (**E-M Algorithm**), sample from, integrate, and convolve:





# SIMULATION: Shortest-Latency Paths

- Developed a **Gaussian Mixture Model** implementation to manipulate arbitrary edge **latency distributions**.
  - I.e. Gaussians are easy to fit (**E-M Algorithm**), sample from, integrate, and convolve:



- Real-world latency distributions can be very difficult to estimate; we may **only know their mean and variance** for example.

# SIMULATION: Shortest-Latency Paths

- Developed **exact solutions** for the **distribution** on the latency of the shortest path through the stochastic network, with a generalization to the **shortest-flow**.

$$1 - \sum_{b=0}^{2^n-1} P(\wp = b) \prod_{\wp_i \in b} \left[ 1 - P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right) \right]$$

# SIMULATION: Shortest-Latency Paths

- Developed **exact solutions** for the **distribution** on the latency of the shortest path through the stochastic network, with a generalization to the **shortest-flow**.

$$1 - \sum_{b=0}^{2^n-1} P(\wp = b) \prod_{\wp_i \in b} \left[ 1 - P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right) \right]$$

☹ Exponential Time

Rapidly estimated with a Bayesian Network

Convolution of Edge Latency Distributions



# SIMULATION: Shortest-Latency Paths

- Developed **exact solutions** for the **distribution** on the latency of the shortest path through the stochastic network, with a generalization to the **shortest-flow**.

The diagram illustrates the exact solution for the distribution of latency on the shortest path. The formula is:

$$1 - \sum_{b=0}^{2^n-1} P(\wp = b) \prod_{\wp_i \in b} \left[ 1 - P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right) \right]$$

Annotations in yellow boxes:

- ⊗ Exponential Time**: Points to the summation term  $\sum_{b=0}^{2^n-1}$ .
- Rapidly estimated with a Bayesian Network**: Points to the probability term  $P(\wp = b)$ .
- Convolution of Edge Latency Distributions**: Points to the inner probability term  $P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right)$ .

- Developed a **Taylor Series approximation** for connectivity under very low-latency conditions.

# SIMULATION: Shortest-Latency Paths

- Developed **exact solutions** for the **distribution** on the latency of the shortest path through the stochastic network, with a generalization to the **shortest-flow**.

The diagram illustrates the formula for the distribution of shortest path latency. The formula is:

$$1 - \sum_{b=0}^{2^n-1} P(\wp = b) \prod_{\wp_i \in b} \left[ 1 - P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right) \right]$$

Callouts explain the components of the formula:

- ⊗ Exponential Time**: Points to the summation term  $\sum_{b=0}^{2^n-1}$ .
- Rapidly estimated with a Bayesian Network**: Points to the probability term  $P(\wp = b)$ .
- Convolution of Edge Latency Distributions**: Points to the inner probability term  $P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right)$ .

- Developed a **Taylor Series approximation** for connectivity under very low-latency conditions.
- Simple, non-stratified, non-recursive **Monte Carlo Simulation** outperformed all other approaches in terms of **speed and variance**, and thus was used going forward.

# SIMULATION: Shortest-Latency Paths

- Developed **exact solutions** for the **distribution** on the latency of the shortest path through the stochastic network, with a generalization to the **shortest-flow**.

The diagram illustrates the formula for the distribution of the shortest path latency in a stochastic network. The formula is:

$$1 - \sum_{b=0}^{2^n-1} P(\wp = b) \prod_{\wp_i \in b} \left[ 1 - P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right) \right]$$

Annotations in the diagram:

- A callout box labeled "⊙ Exponential Time" points to the summation term  $\sum_{b=0}^{2^n-1}$ .
- A callout box labeled "Rapidly estimated with a Bayesian Network" points to the probability term  $P(\wp = b)$ .
- A callout box labeled "Convolution of Edge Latency Distributions" points to the inner probability term  $P \left( \sum_{e_j \in \wp_i} L_{e_j} \leq x \right)$ .

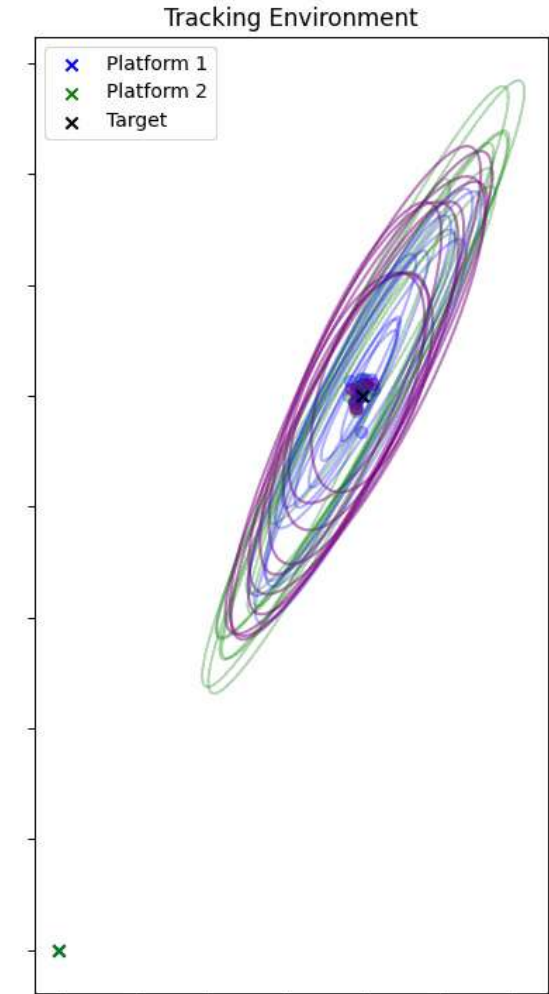
- Developed a **Taylor Series approximation** for connectivity under very low-latency conditions.
- Simple, non-stratified, non-recursive **Monte Carlo Simulation** outperformed all other approaches in terms of **speed and variance**, and thus was used going forward.
  - Simple + Parallelizable: Randomly sample edge failures & edge latencies, then run Dijkstra.
  - (Gibbs Sampling in a **Bayesian Network** can be much more efficient for certain problems)

# Types of Networks: Fusion Networks

- **Target, platform, sensor, data-fusion,** and **weapon** nodes emulating Kill-Chain track-sharing.
- Components have a **track-quality distribution** describing the **uncertainty** in tracks being passed into and out of fusion nodes.

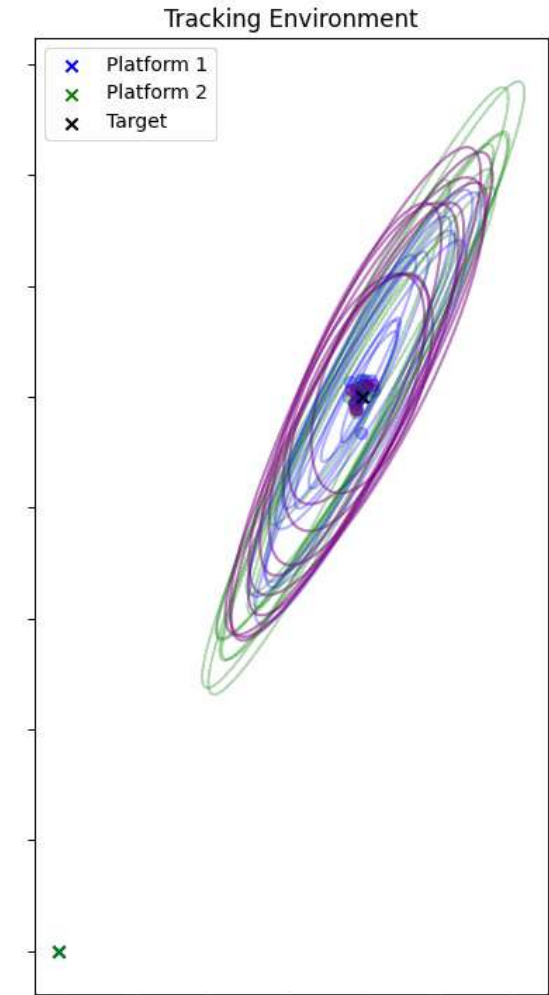
# Types of Networks: Fusion Networks

- **Target, platform, sensor, data-fusion, and weapon** nodes emulating Kill-Chain track-sharing.
- Components have a **track-quality distribution** describing the **uncertainty** in tracks being passed into and out of fusion nodes.



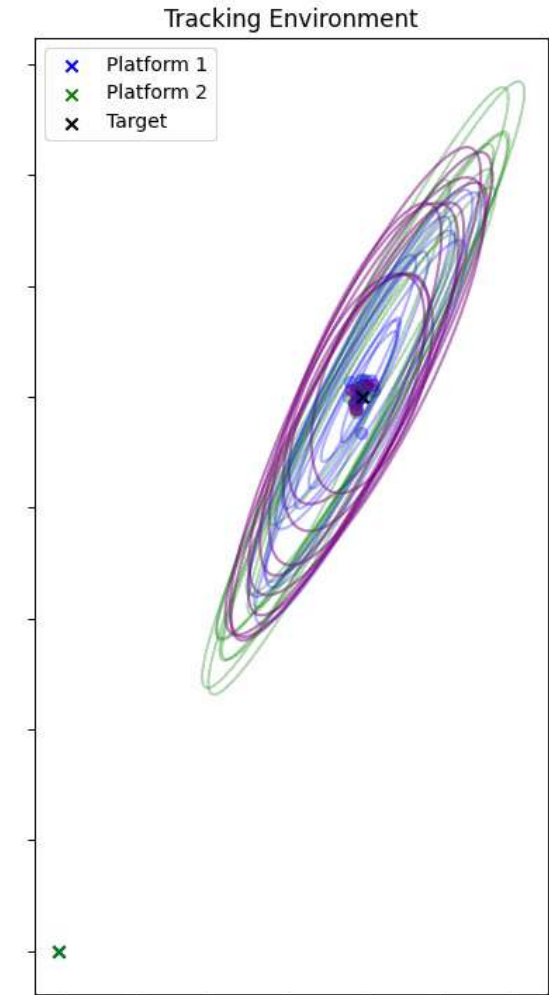
# Types of Networks: Fusion Networks

- **Target, platform, sensor, data-fusion, and weapon** nodes emulating Kill-Chain track-sharing.
- Components have a **track-quality distribution** describing the **uncertainty** in tracks being passed into and out of fusion nodes.
- *Check out my July 1<sup>st</sup> Scrum Talk for many more details on fusion parameters and simulation!*



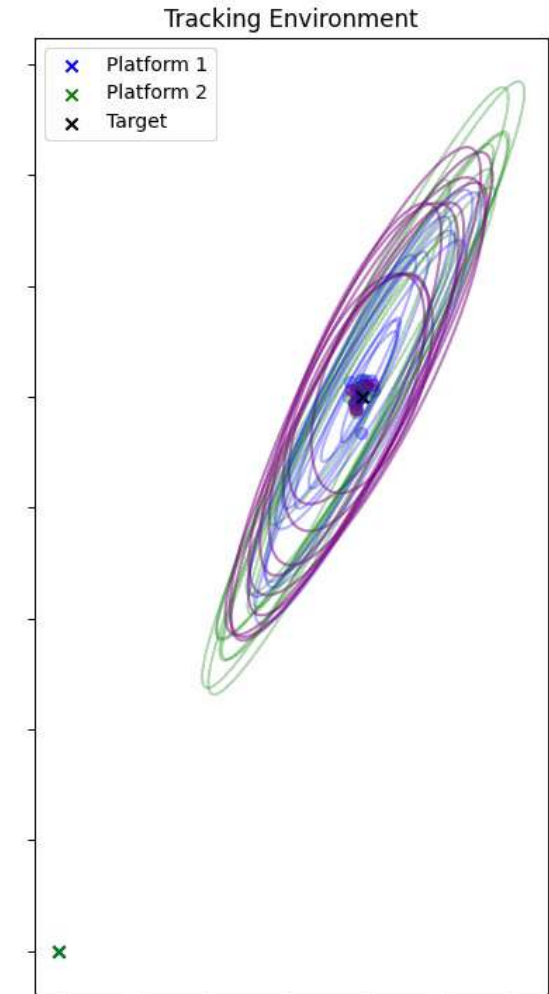
# Types of Networks: Fusion Networks

- **Target, platform, sensor, data-fusion, and weapon** nodes emulating Kill-Chain track-sharing.
- Components have a **track-quality distribution** describing the **uncertainty** in tracks being passed into and out of fusion nodes.
  - *Check out my July 1<sup>st</sup> Scrum Talk for many more details on fusion parameters and simulation!*
- Developed **Bayesian Network** simulations for **time-independent** fusion.
  - Implemented in Pgmpy for **fast** approximate inference over possible input-to-output paths.



# Types of Networks: Fusion Networks

- **Target, platform, sensor, data-fusion, and weapon** nodes emulating Kill-Chain track-sharing.
- Components have a **track-quality distribution** describing the **uncertainty** in tracks being passed into and out of fusion nodes.
  - *Check out my July 1<sup>st</sup> Scrum Talk for many more details on fusion parameters and simulation!*
- Developed **Bayesian Network** simulations for **time-independent** fusion.
  - Implemented in Pgmpy for **fast** approximate inference over possible input-to-output paths.
- Developed an **Agent-Based** simulation for **time-dependent** track flow.
  - Numba-accelerated for faster runtimes, returning a track-quality distribution as a **function of time**

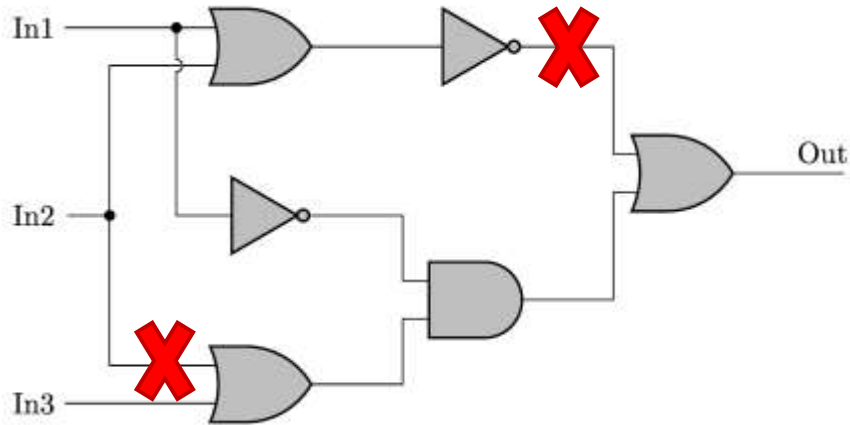




# Types of Networks: Booleans

## Boolean Satisfiability Networks:

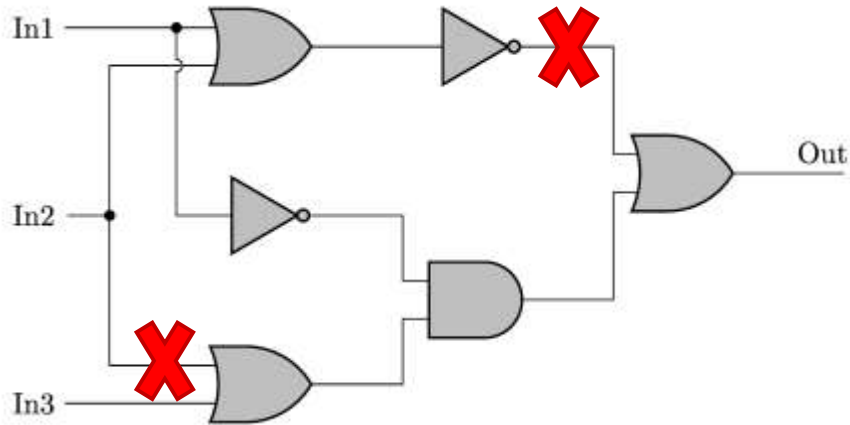
- Nodes must be **satisfied** by combinations of different incoming **signals**.
- **Ex:** Weapon X requires authorization Y *AND* {sensor modality Z *OR* modality W}



# Types of Networks: Booleans

## Boolean Satisfiability Networks:

- Nodes must be **satisfied** by combinations of different incoming **signals**.
- **Ex:** Weapon X requires authorization Y *AND* {sensor modality Z *OR* modality W}

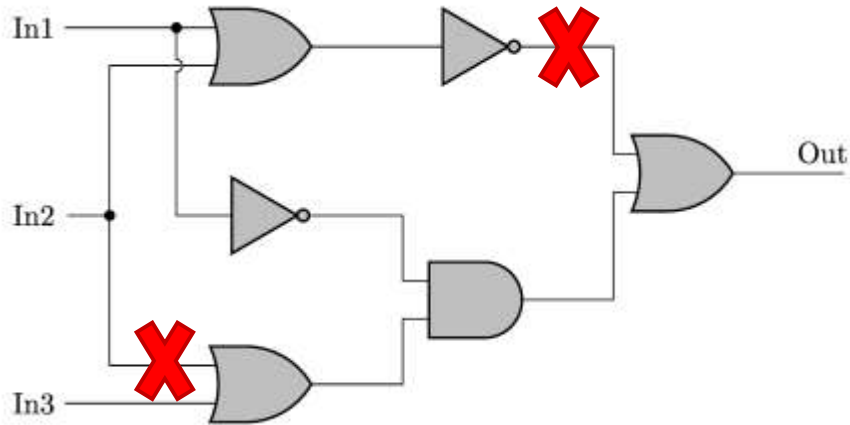


Electrical circuits only have 1 “type” of signal, whereas a **mission thread** has many “types”: Command & Control, Sensor Modalities, Resource Flow, Physical Projectiles etc.

# Types of Networks: Booleans

## Boolean Satisfiability Networks:

- Nodes must be **satisfied** by combinations of different incoming **signals**.
- **Ex:** Weapon X requires authorization Y *AND* {sensor modality Z *OR* modality W}



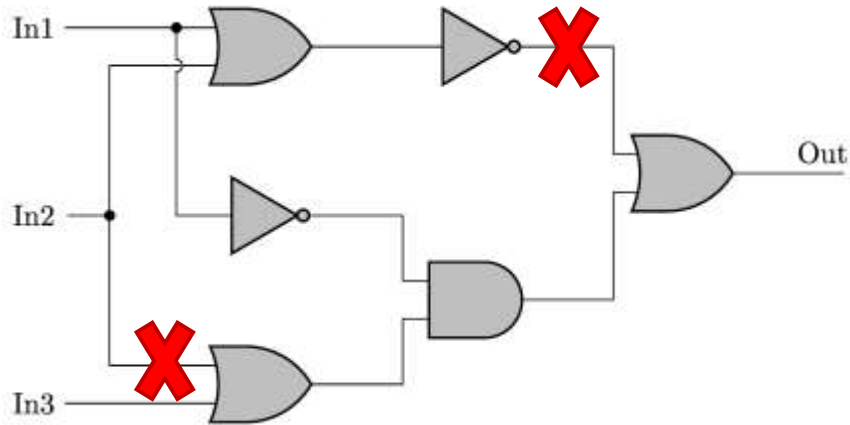
Electrical circuits only have 1 “type” of signal, whereas a **mission thread** has many “types”: Command & Control, Sensor Modalities, Resource Flow, Physical Projectiles etc.

What is the optimal means of **configuring our components** into a system that satisfies all of our **requirements**?

# Types of Networks: Booleans

## Boolean Satisfiability Networks:

- Nodes must be **satisfied** by combinations of different incoming **signals**.
- **Ex:** Weapon X requires authorization Y *AND* {sensor modality Z *OR* modality W}



Electrical circuits only have 1 “type” of signal, whereas a **mission thread** has many “types”: Command & Control, Sensor Modalities, Resource Flow, Physical Projectiles etc.

What is the optimal means of **configuring our components** into a system that satisfies all of our **requirements**?

- Adapted for **mission planning**: output Boolean is the **success**, edge **failures** (X) are thwarted plans, **nodes** are our assets / processes, etc.

# SIMULATION of a Boolean Network:

- Generate a graph of **all possible component connections**: our goal is a feasible **subgraph**.
  - *If  $X$  emits signal “ $a$ ” and  $Y$  receives signal “ $a$ ”, then join  $(X,Y)$*

# SIMULATION of a Boolean Network:

- Generate a graph of **all possible component connections**: our goal is a feasible **subgraph**.
  - *If  $X$  emits signal “ $a$ ” and  $Y$  receives signal “ $a$ ”, then join  $(X,Y)$*
- **User-inputted** SymPy expressions control node logical behavior and signal **requirements**.

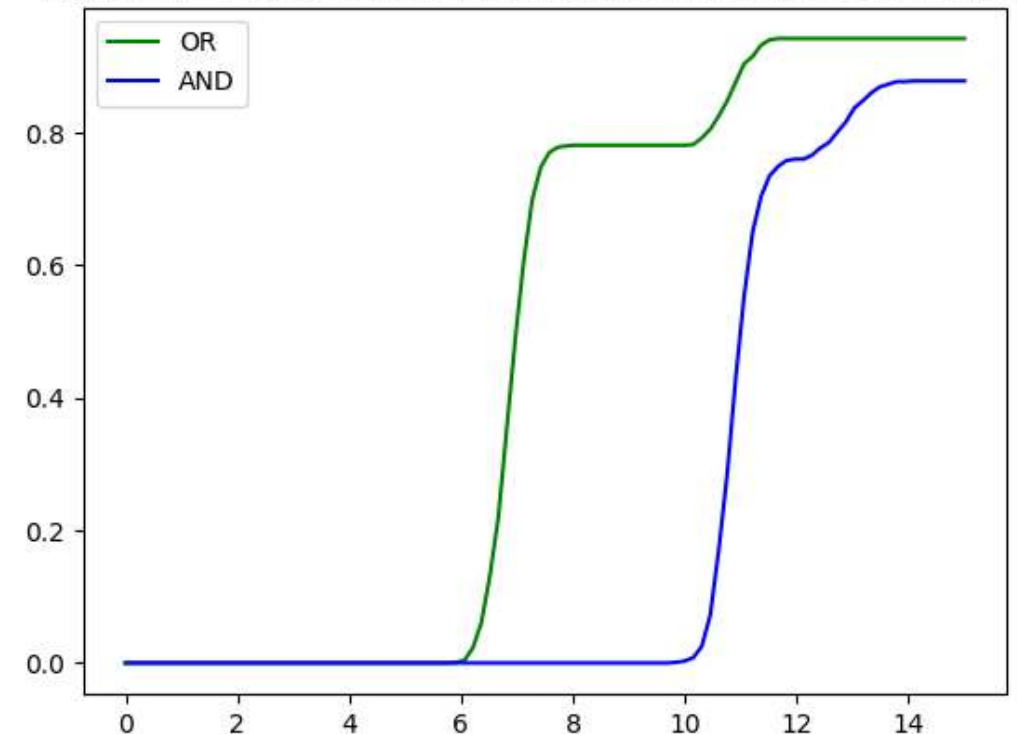
# SIMULATION of a Boolean Network:

- Generate a graph of **all possible component connections**: our goal is a feasible **subgraph**.
  - *If  $X$  emits signal “ $a$ ” and  $Y$  receives signal “ $a$ ”, then join  $(X,Y)$*
- **User-inputted** SymPy expressions control node logical behavior and signal **requirements**.
- Use PySAT **MaxSAT solver** or **0-1 Integer Linear Program** to determine a minimum-cost signal flow / **routing**.
  - Generate a (integer) **polytope** for performant networks.

# SIMULATION of a Boolean Network:

- Generate a graph of **all possible component connections**: our goal is a feasible **subgraph**.
  - *If  $X$  emits signal “ $a$ ” and  $Y$  receives signal “ $a$ ”, then join  $(X,Y)$*
- **User-inputted** SymPy expressions control node logical behavior and signal **requirements**.
- Use PySAT **MaxSAT solver** or **0-1 Integer Linear Program** to determine a minimum-cost signal flow / **routing**.
  - Generate a (integer) **polytope** for performant networks.
- Cannot solve **dynamic allocation** (eg. assignments to engagements over time), but can analyze **static** changes to network communication **structure**.

P(Latency  $\leq x$ ) w/ Vertex 4 Having an OR vs. AND Gate Behavior



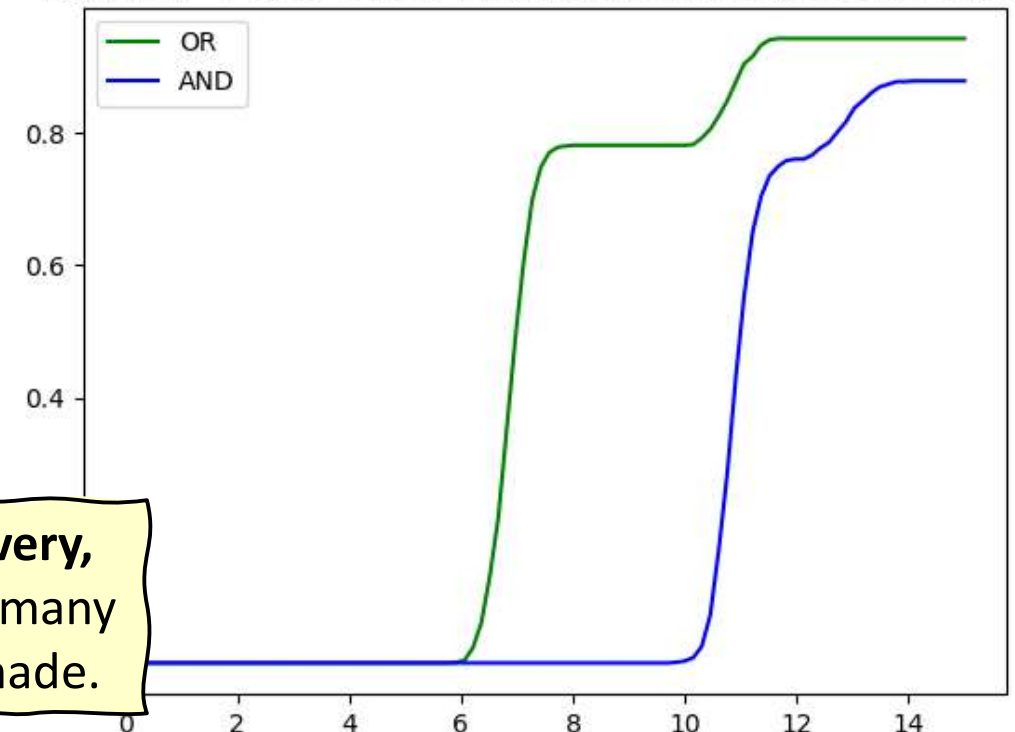


# SIMULATION of a Boolean Network:

- Generate a graph of **all possible component connections**: our goal is a feasible **subgraph**.
  - *If X emits signal “a” and Y receives signal “a”, then join (X,Y)*
- **User-inputted** SymPy expressions control node logical behavior and signal **requirements**.
- Use PySAT **MaxSAT solver** or **0-1 Integer Linear Program** to determine a minimum-cost signal flow / **routing**.
  - Generate a (integer) **polytope** for performant networks.
- Cannot solve **dynamic allocation** (eg. assignments to engagements over time), but can analyze **static** changes to network communication **structure**.

Mission planning is a **very, very hard problem**, so many simplifications were made.

P(Latency  $\leq x$ ) w/ Vertex 4 Having an OR vs. AND Gate Behavior



# The Unreasonable Effectiveness of Linear Models:

**Driving Assumption:** Features which can **best predict** network performance are the features of **greatest importance**.

# The Unreasonable Effectiveness of Linear Models:

**Driving Assumption:** Features which can **best predict** network performance are the features of **greatest importance**.



**Implication:** Use (sparse) linear **regression** to estimate the **combinations of features** with optimal performance.

# The Unreasonable Effectiveness of Linear Models:

**Driving Assumption:** Features which can **best predict** network performance are the features of **greatest importance**.



**Implication:** Use (sparse) linear **regression** to estimate the **combinations of features** with optimal performance.

For “reasonable” performance metrics, linear models perform **remarkably well** in **predicting, explaining, and recommending improvements** to network behavior!

# The Unreasonable Effectiveness of Linear Models:

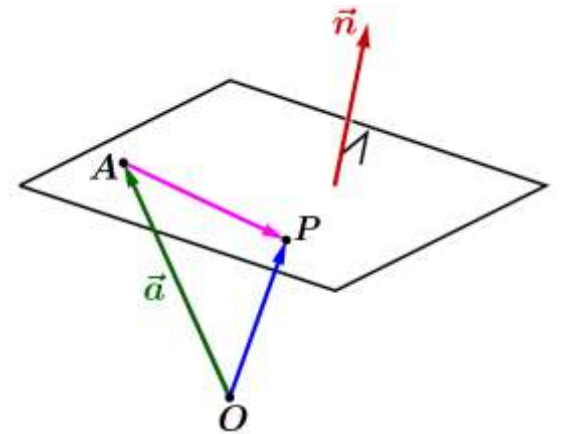
**Driving Assumption:** Features which can **best predict** network performance are the features of **greatest importance**.



**Implication:** Use (sparse) linear **regression** to estimate the **combinations of features** with optimal performance.

## “Gradient” Vector Analysis as a Regression Problem:

- Due to  $\approx$  linearity, dimensionality reduction techniques like PCA and Partial Least Squares (PLS) give **directions of steepest improvement**.



For “reasonable” performance metrics, linear models perform **remarkably well** in **predicting, explaining, and recommending improvements** to network behavior!

# The Unreasonable Effectiveness of Linear Models:

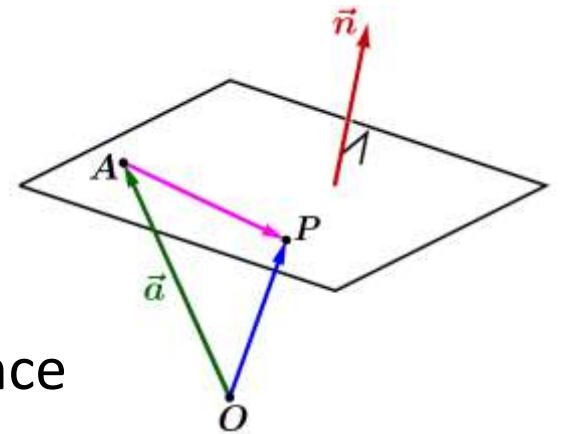
**Driving Assumption:** Features which can **best predict** network performance are the features of **greatest importance**.



**Implication:** Use (sparse) linear **regression** to estimate the **combinations of features** with optimal performance.

## “Gradient” Vector Analysis as a Regression Problem:

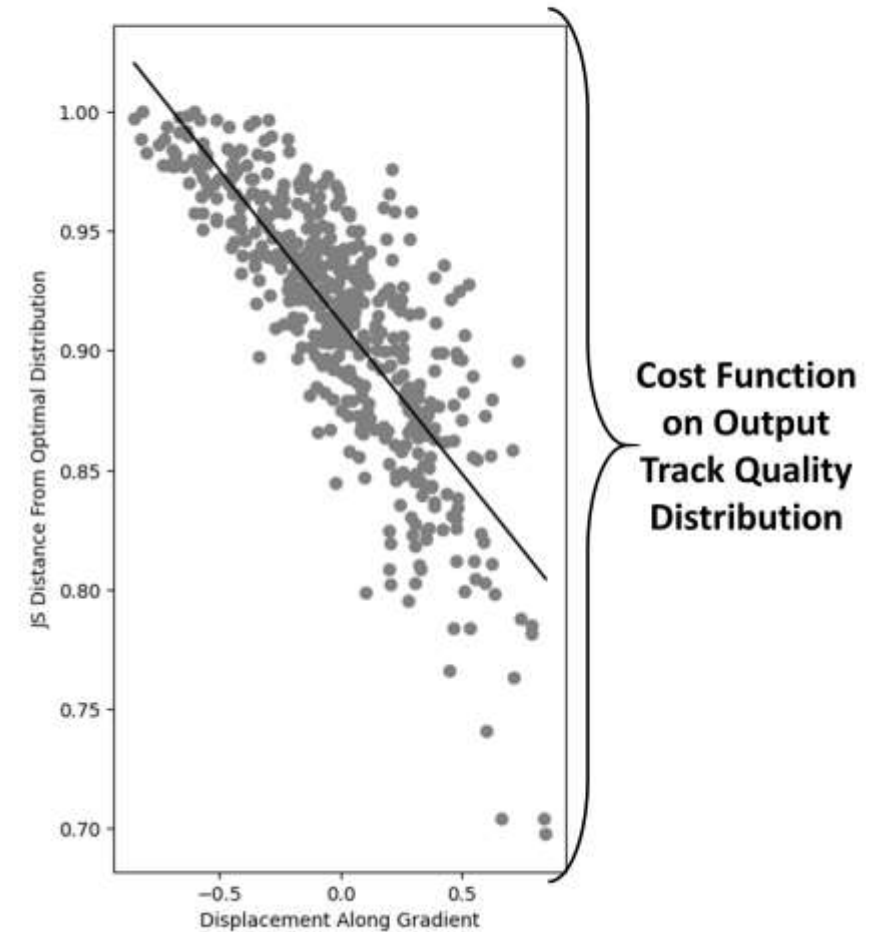
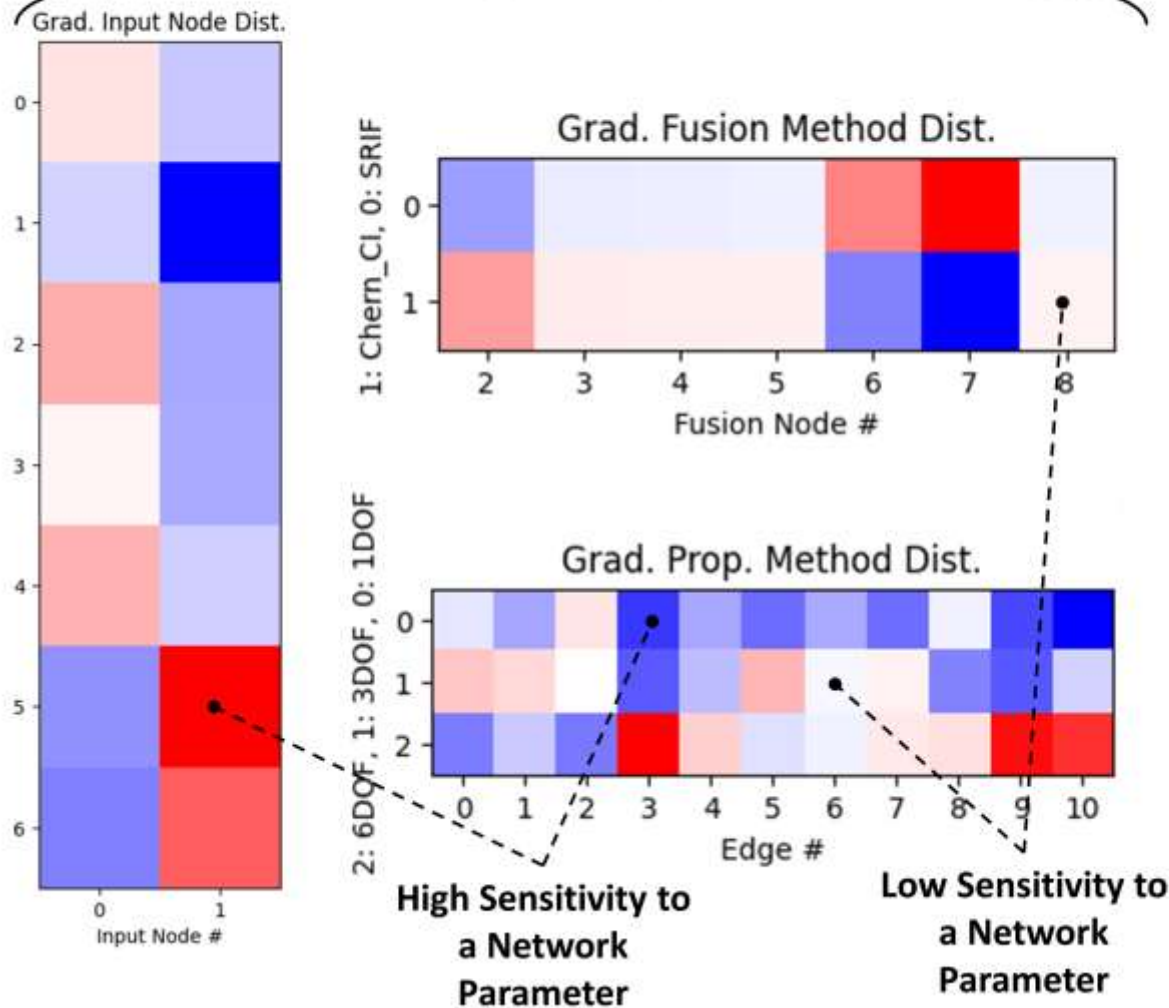
- Due to  $\approx$  linearity, dimensionality reduction techniques like PCA and **Partial Least Squares (PLS)** give **directions of steepest improvement**.
- Represents a **globally optimal** axis strongly **correlated** with performance improvement considering **all features** simultaneously.
  - ***Superadditivity of Network Performance, See Braess’ Paradox***



For “reasonable” performance metrics, linear models perform **remarkably well** in **predicting, explaining, and recommending improvements** to network behavior!

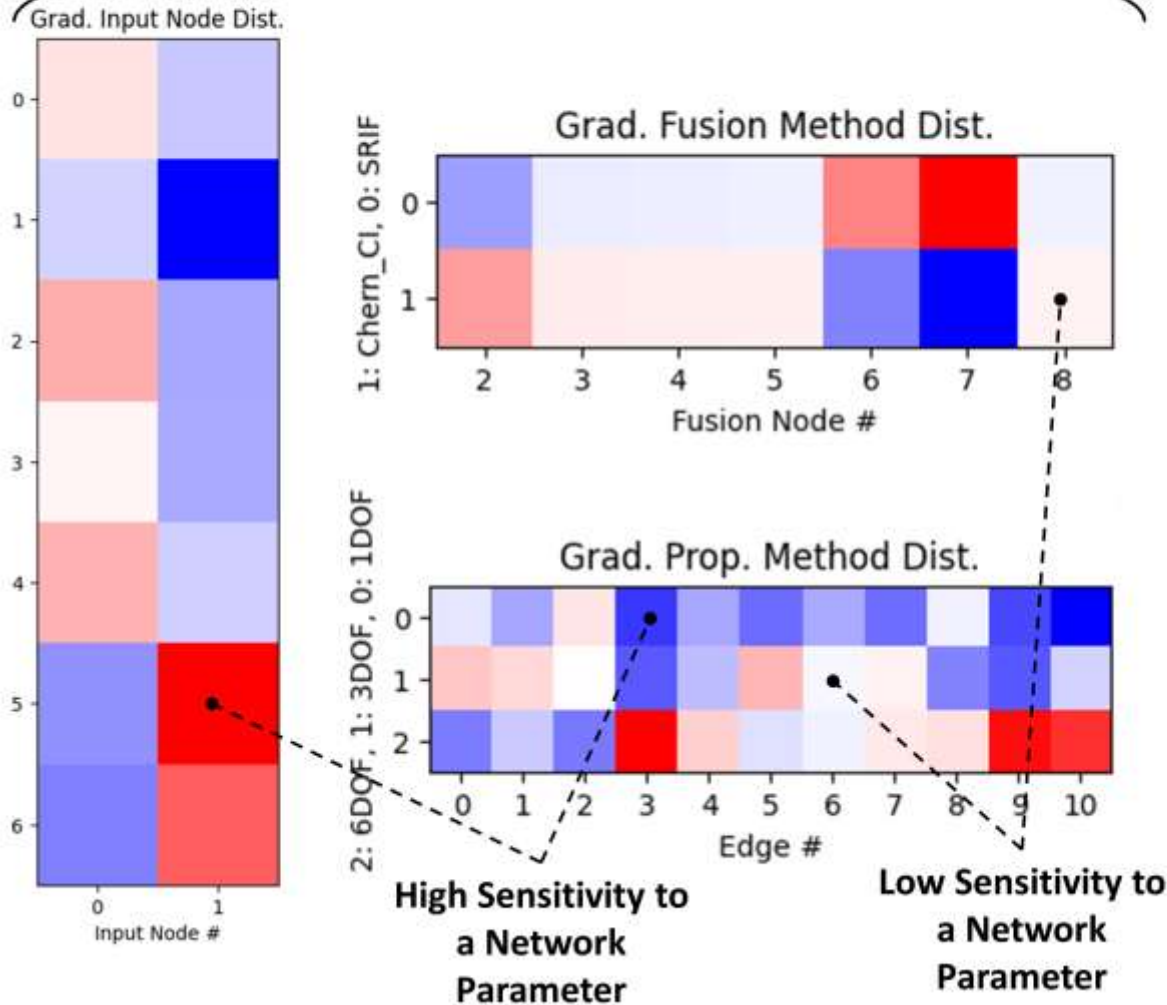
# The Unreasonable Effectiveness of Linear Models:

Data-Driven Heatmap of how Influential Network Parameters are to the Overall Performance

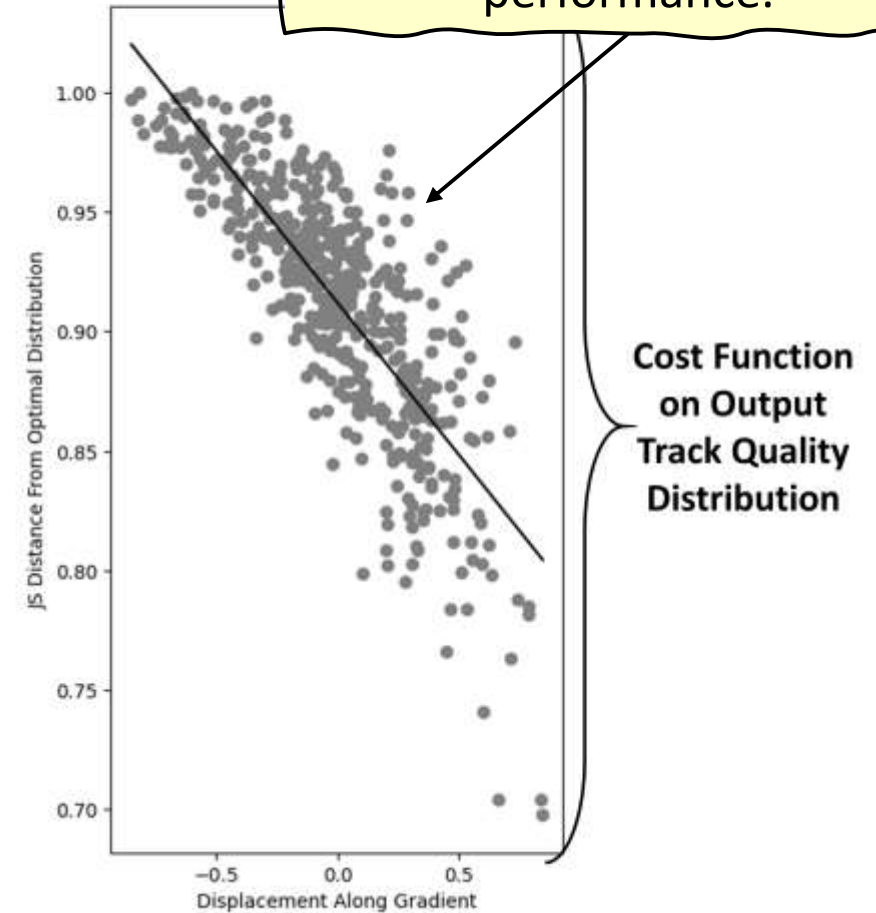


# The Unreasonable Effectiveness of Linear Models:

Data-Driven Heatmap of how Influential Network Parameters are to the Overall Performance



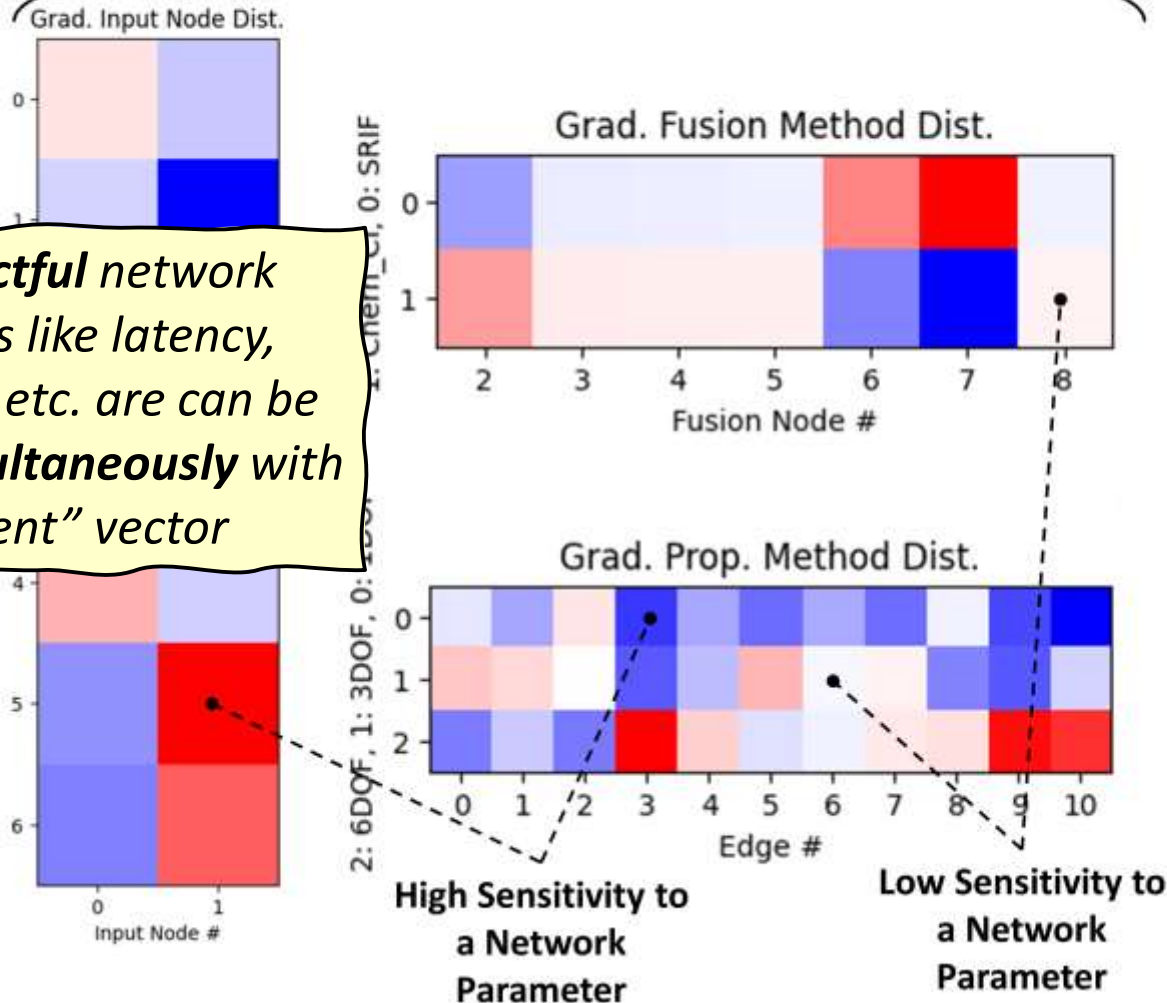
Notice the approximate **linearity** of a quite **complex** tracking network's performance.





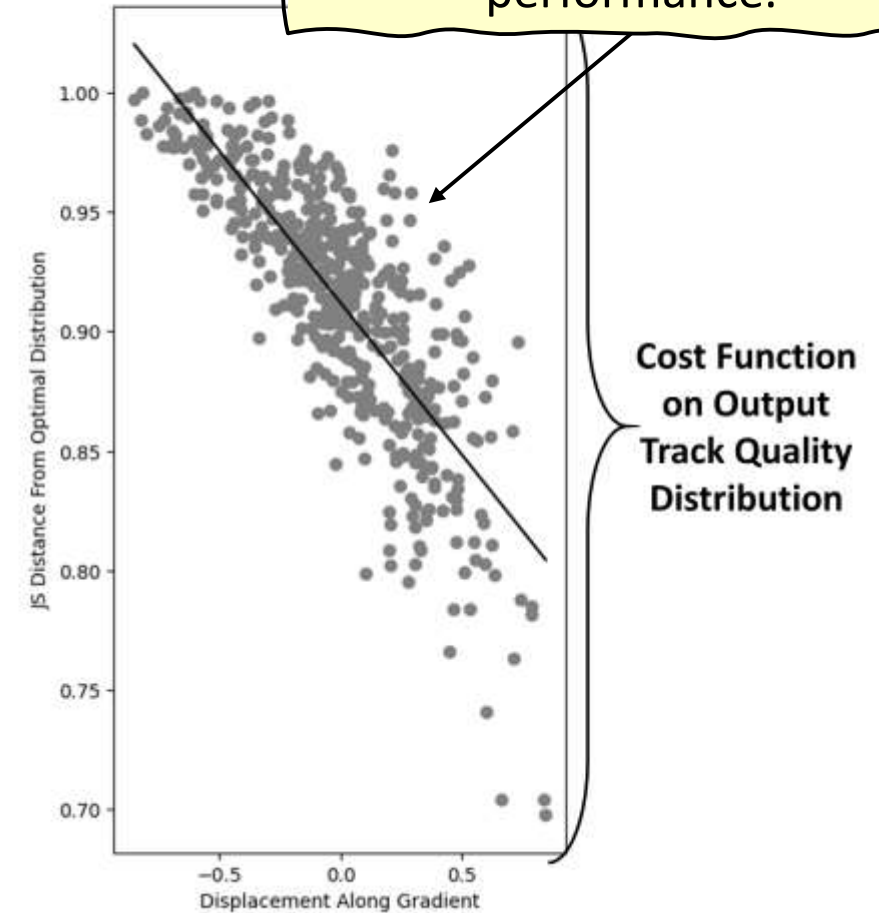
# The Unreasonable Effectiveness of Linear Models:

Data-Driven Heatmap of how Influential Network Parameters are to the Overall Performance



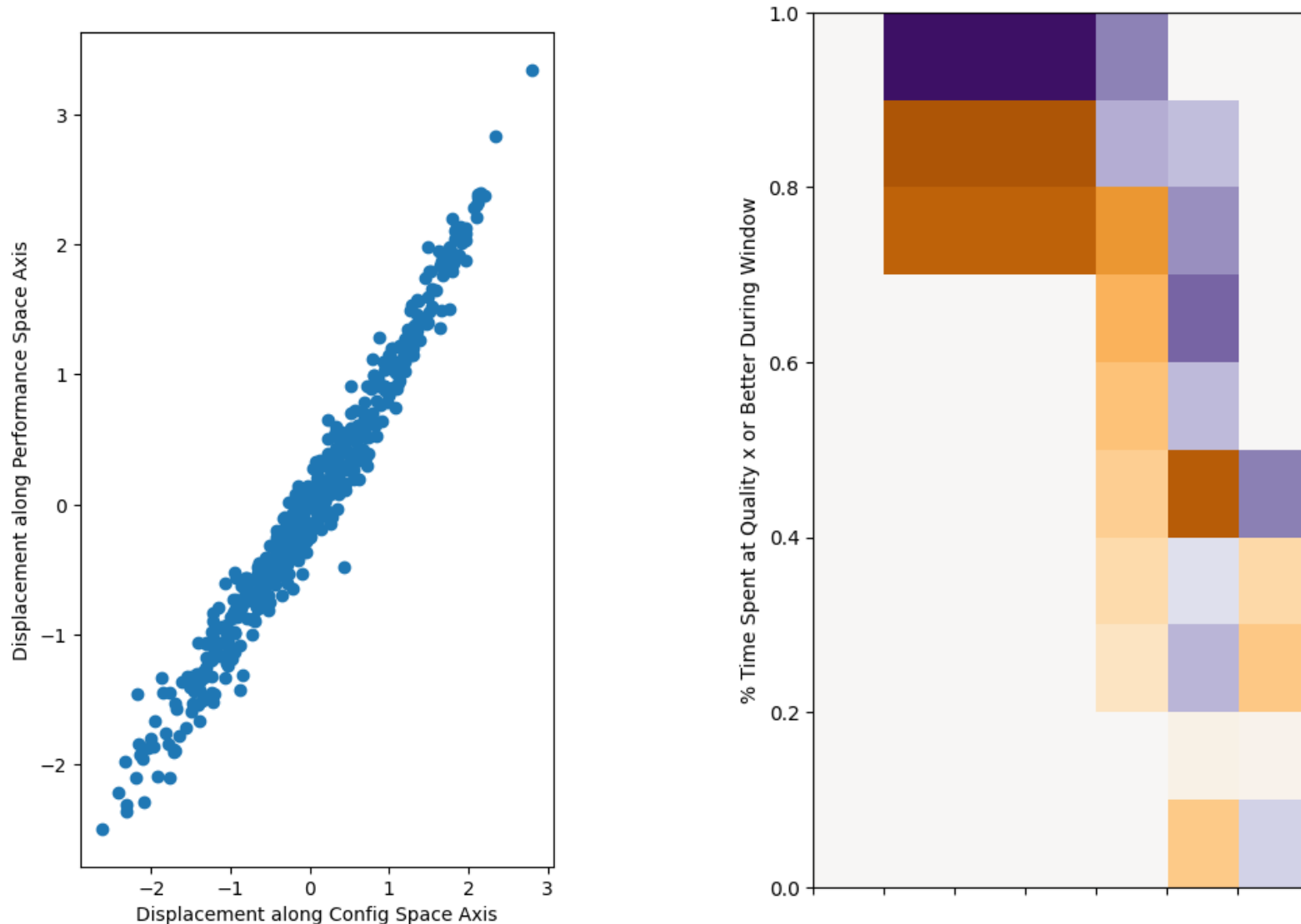
How **impactful** network parameters like latency, connectivity, etc. are can be captured **simultaneously** with a “gradient” vector

Notice the approximate **linearity** of a quite **complex** tracking network’s performance.



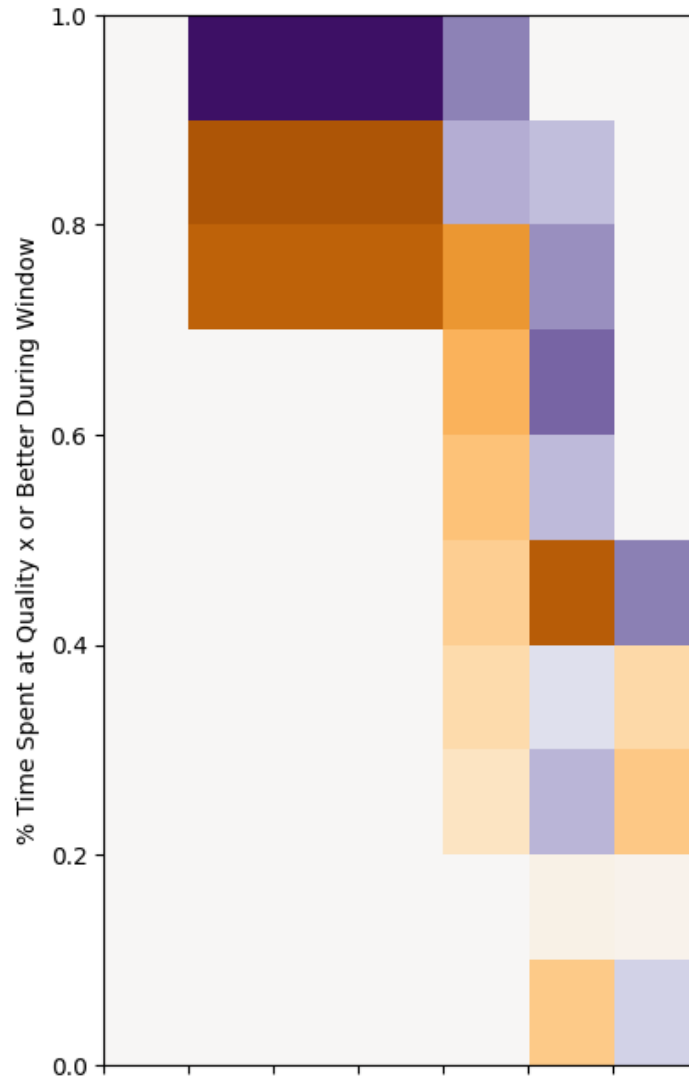
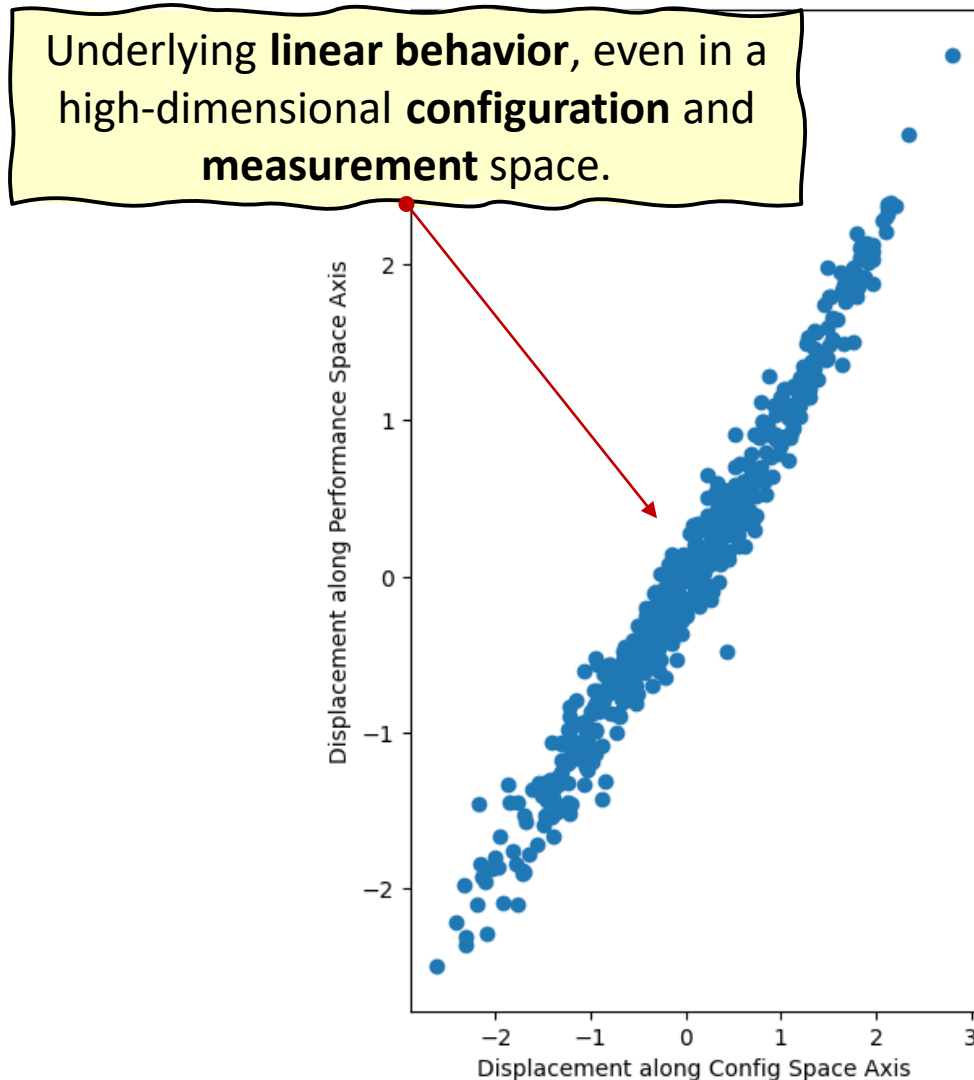
# Linear Models: Eigen-Measurements

Linearizing **both** the input and output features can provide dominant “axes of performance variance” to **accompany** “axes of steepest ascent”



# Linear Models: Eigen-Measurements

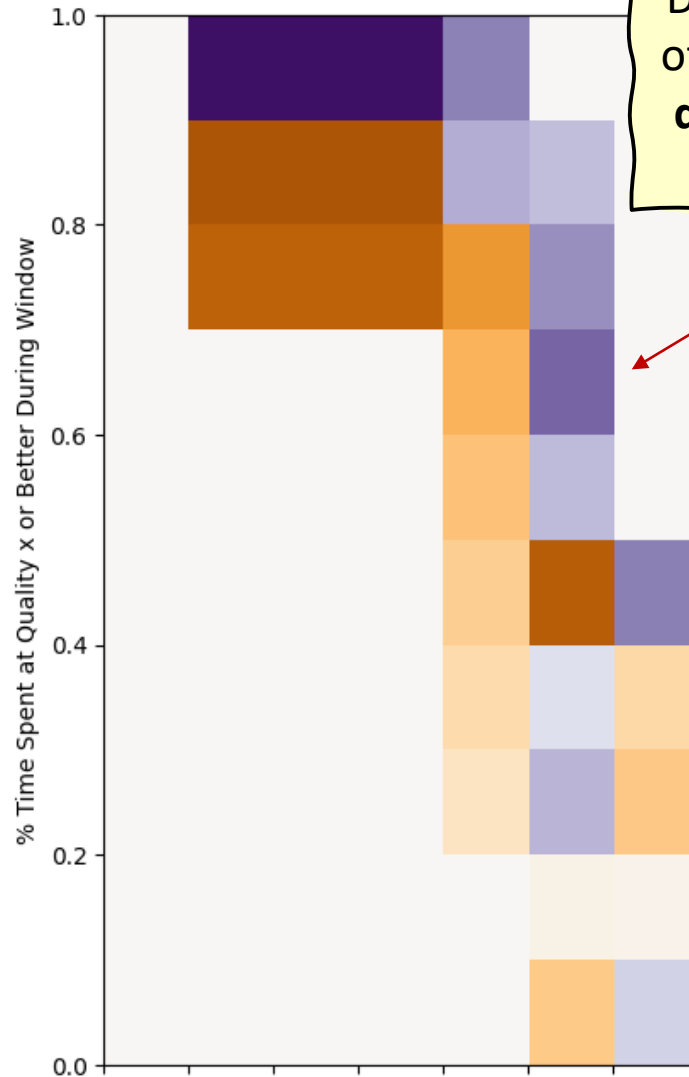
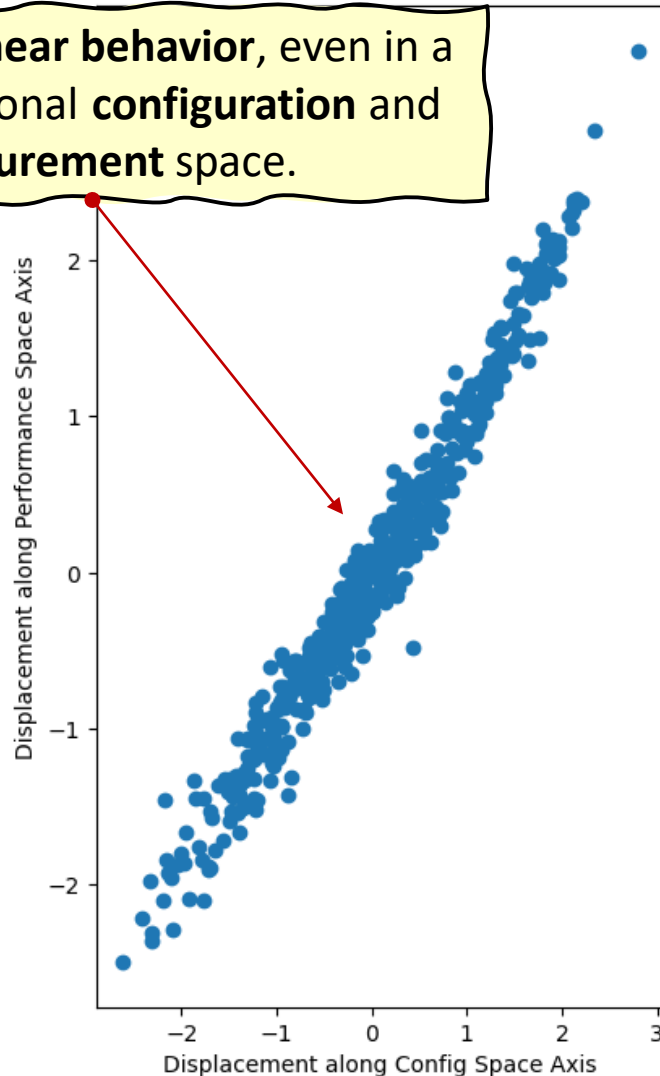
Linearizing **both** the input and output features can provide dominant “axes of performance variance” to **accompany** “axes of steepest ascent”



# Linear Models: Eigen-Measurements

Linearizing **both** the input and output features can provide dominant “axes of performance variance” to **accompany** “axes of steepest ascent”

Underlying **linear behavior**, even in a high-dimensional **configuration** and **measurement** space.

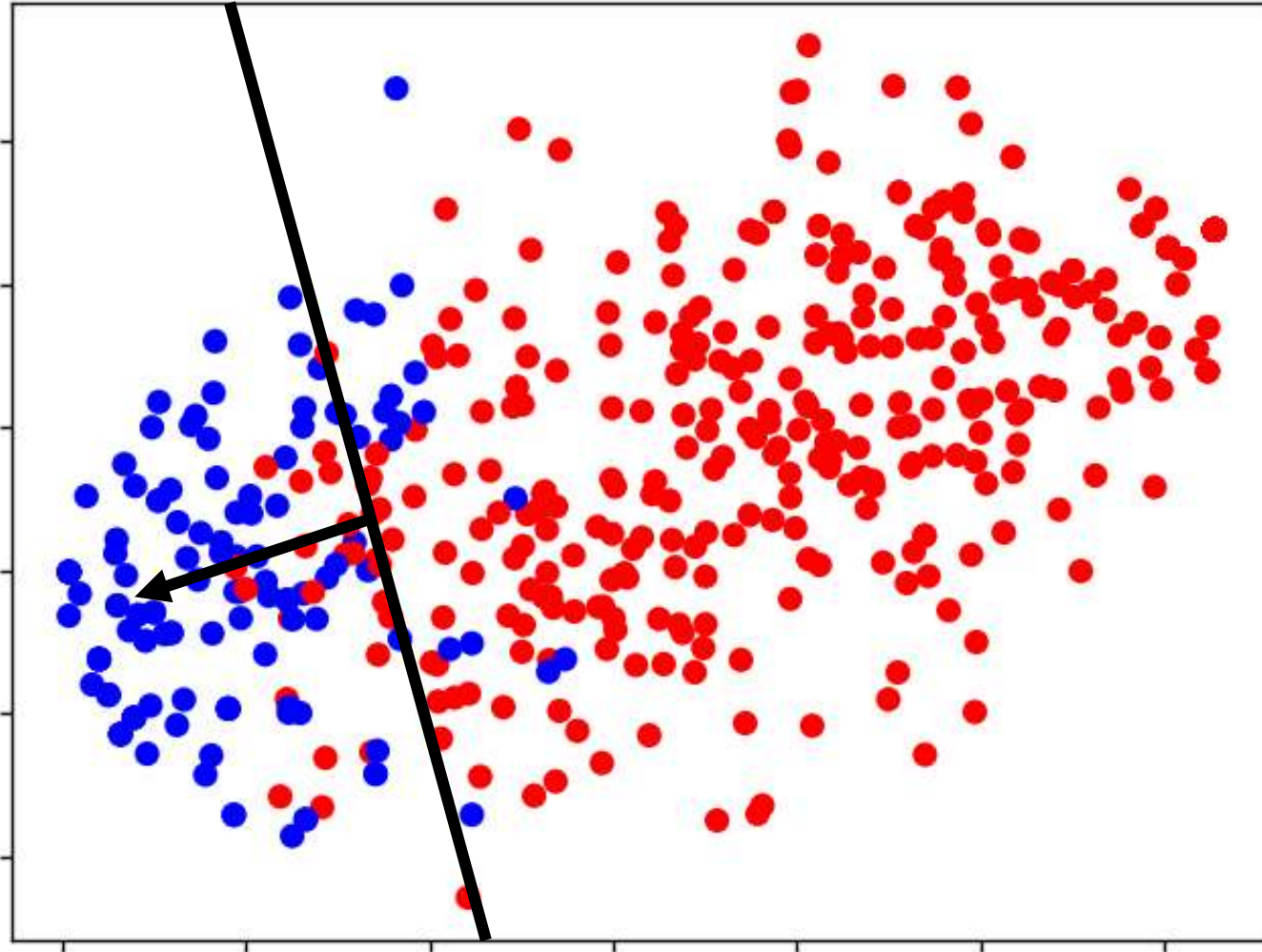


Data-driven “**Eigenface**” of how the **performance distribution** varies with network parameters.

# Linear Models: Discrete Optimization

Here's an example of a Boolean network that maps **binary** inputs (ie if components are connected) to a **binary** success flag.

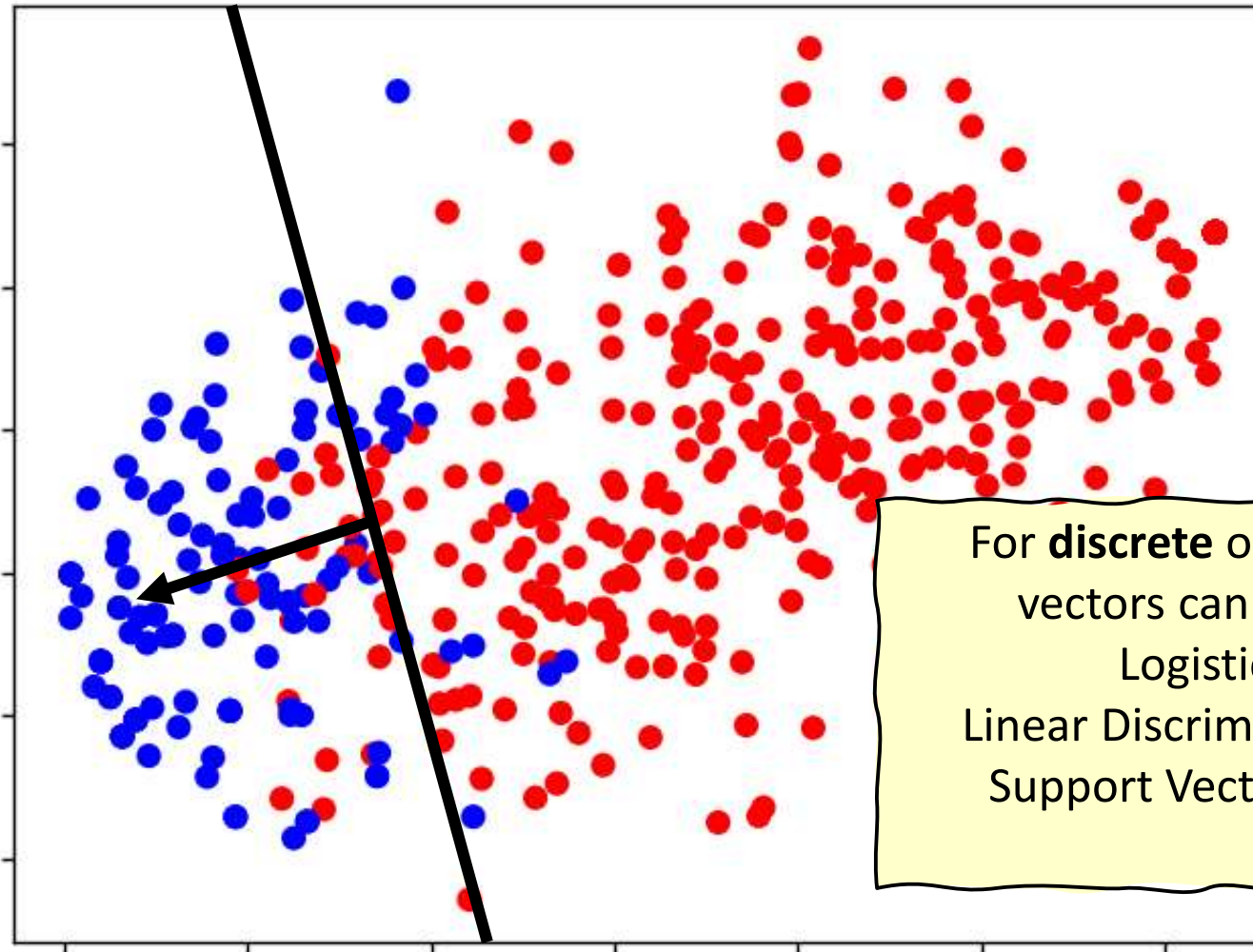
- Linear models still seem to make accurate predictions.



# Linear Models: Discrete Optimization

Here's an example of a Boolean network that maps **binary** inputs (ie if components are connected) to a **binary** success flag.

- Linear models still seem to make accurate predictions.



For **discrete** outcomes, “gradient” vectors can be computed via:  
Logistic Regression,  
Linear Discriminant Analysis (LDA),  
Support Vector Machines (SVM)

...

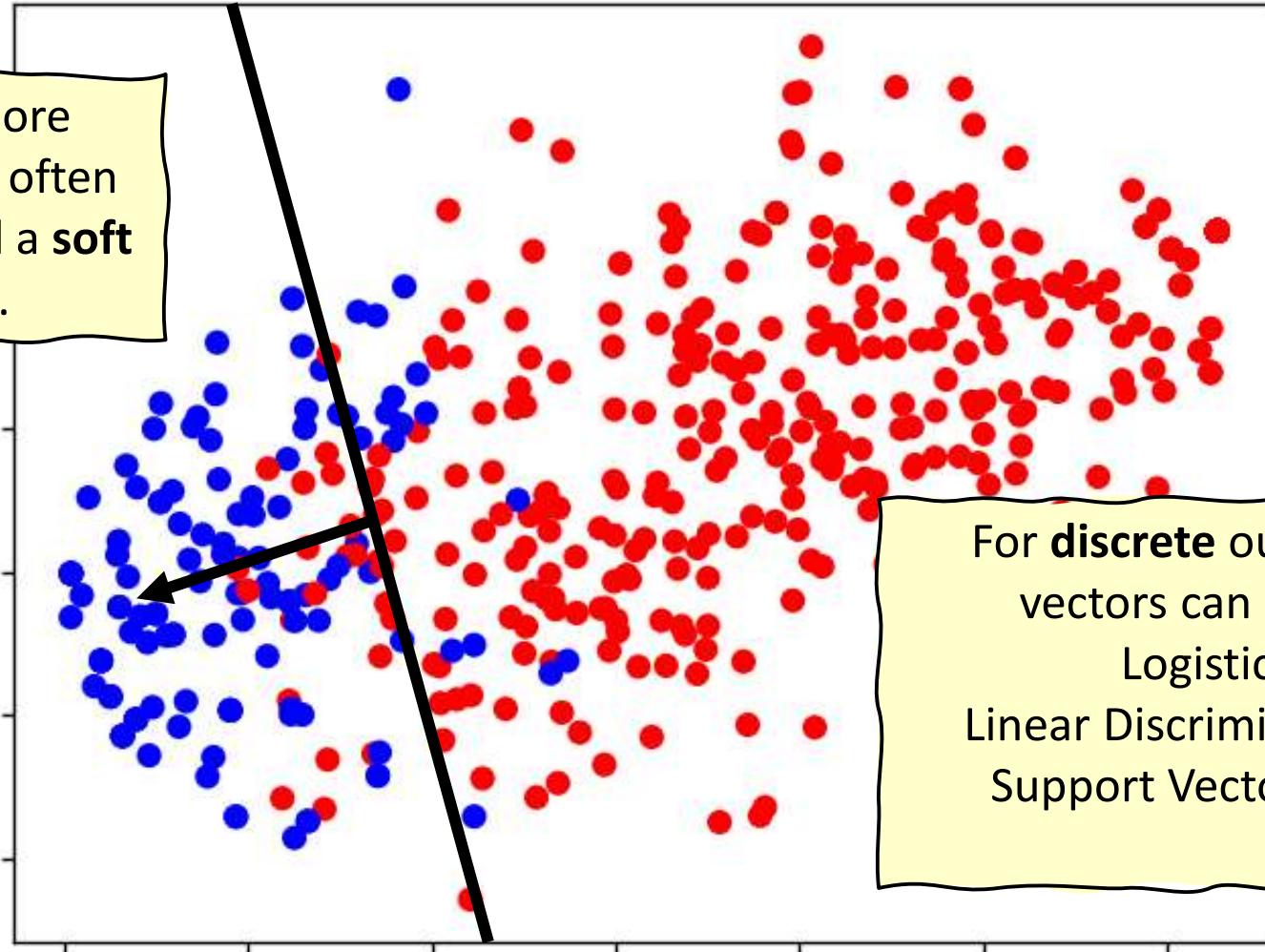


# Linear Models: Discrete Optimization

Here's an example of a Boolean network that maps **binary** inputs (ie if components are connected) to a **binary** success flag.

- Linear models still seem to make accurate predictions.

For a **smoother** and more insightful regression, we often use **soft** inputs  $P(x_i)$  and a **soft** output  $P(\text{Success})$ .



For **discrete** outcomes, “gradient” vectors can be computed via:  
Logistic Regression,  
Linear Discriminant Analysis (LDA),  
Support Vector Machines (SVM)

...

# Linear Models: Sparse Recommendations

**$L_1$  Regression for Parameter Improvement Recommendations:**

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.



# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.

# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.
- Off-the-shelf sparsity-promotion using the L1 penalty in **LASSO / LARS**.

$$\min_{|x|_1 \leq \alpha} \frac{1}{2n} \left\| \begin{bmatrix} A \\ \lambda c^\top \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right\|_2^2$$

# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.
- Off-the-shelf sparsity-promotion using the L1 penalty in **LASSO / LARS**.

“Give me only a **small number** of changes”

$$\min_{|x|_1 \leq \alpha} \frac{1}{2n} \left\| \begin{bmatrix} A \\ \lambda c^\top \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right\|_2^2$$

# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.
- Off-the-shelf sparsity-promotion using the L1 penalty in **LASSO / LARS**.

“Give me only a **small number** of changes”

$$\min_{|x|_1 \leq \alpha} \frac{1}{2n} \left\| \begin{bmatrix} A \\ \lambda c^\top \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right\|_2^2$$

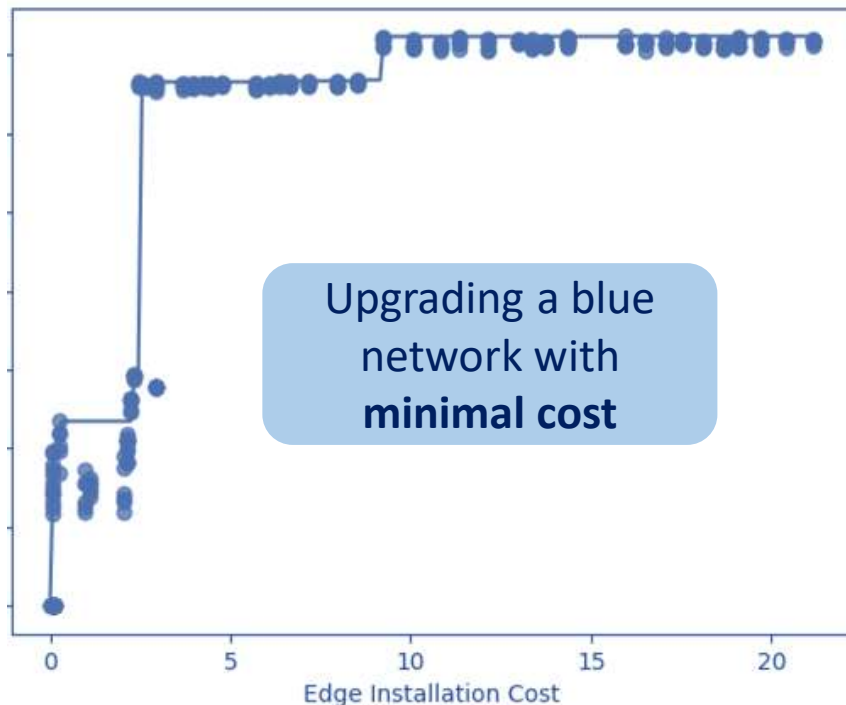
Re-simulate these  
**recommendations** to obtain  
**Pareto-efficient** solutions (best  
performance within budget)

# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.
- Off-the-shelf sparsity-promotion using the L1 penalty in **LASSO / LARS**.

“Give me only a **small number** of changes”



$$\min_{|x|_1 \leq \alpha} \frac{1}{2n} \left| \begin{bmatrix} A \\ \lambda c^\top \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right|_2^2$$

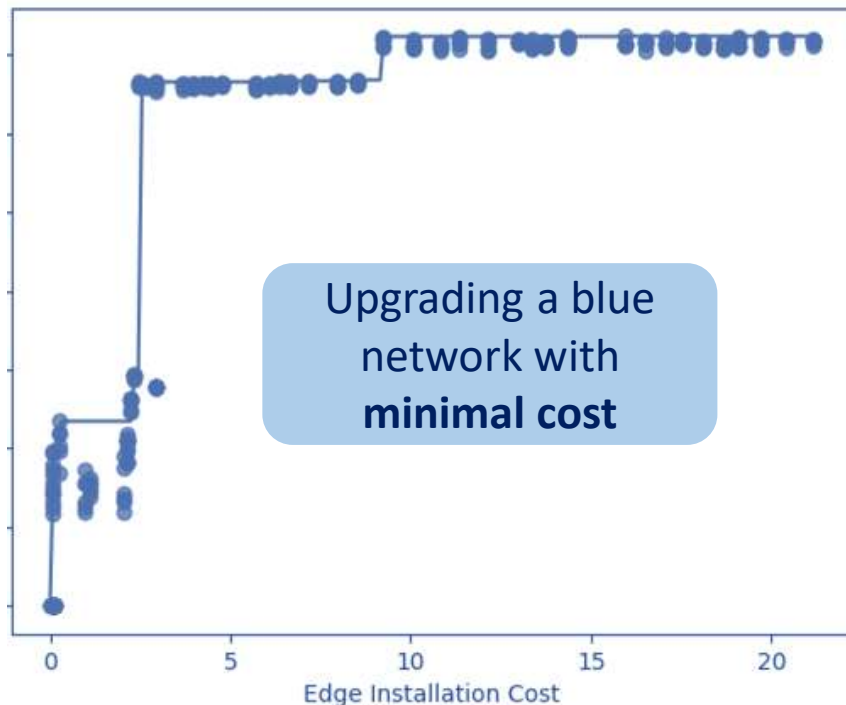
Re-simulate these **recommendations** to obtain **Pareto-efficient** solutions (best performance within budget)

# Linear Models: Sparse Recommendations

## $L_1$ Regression for Parameter Improvement Recommendations:

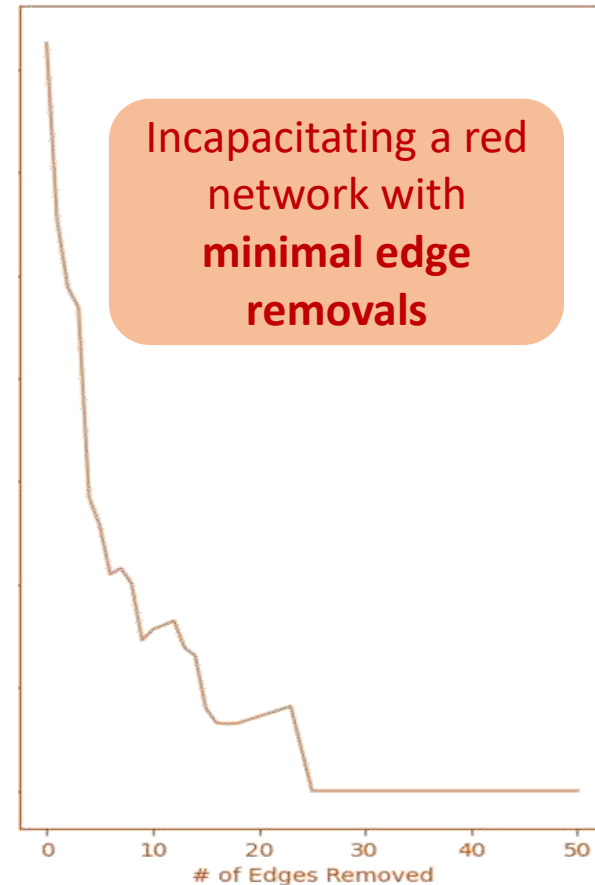
- Exponential # of possible **combinations** of graph parameters to consider for adjustment.
- **Harness linearity** to recommend **parameters** to robustify or **edges** to install / remove; subject to a finite **cost** constraint.
- Off-the-shelf sparsity-promotion using the L1 penalty in **LASSO / LARS**.

"Give me only a **small number** of changes"



$$\min_{|x|_1 \leq \alpha} \frac{1}{2n} \left| \begin{bmatrix} A \\ \lambda c^\top \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right|_2^2$$

Re-simulate these **recommendations** to obtain **Pareto-efficient** solutions (best performance within budget)



# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$



# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Assume a **linear**  
performance response

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Promote **sparse**  
solutions

Assume a **linear**  
performance response

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Promote **sparse**  
solutions

Assume a **linear**  
performance response

Data-driven directions of  
**steepest ascent** found from  
**random sampling**.

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Functional requirements, network capacities, etc.

Promote **sparse** solutions

Assume a **linear** performance response

Data-driven directions of **steepest ascent** found from **random sampling**.

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Functional requirements, network capacities, etc.

Promote **sparse** solutions

Assume a **linear** performance response

Data-driven directions of **steepest ascent** found from **random sampling**.

$$x^*(c) = \arg \max_{\lambda_1, \lambda_2, \dots, \text{Cost}[\hat{x}] \leq c} \text{Score}[\hat{x}(\lambda_1, \lambda_2, \dots)]$$

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Functional requirements, network capacities, etc.

Promote **sparse** solutions

Assume a **linear** performance response

Data-driven directions of **steepest ascent** found from **random sampling**.

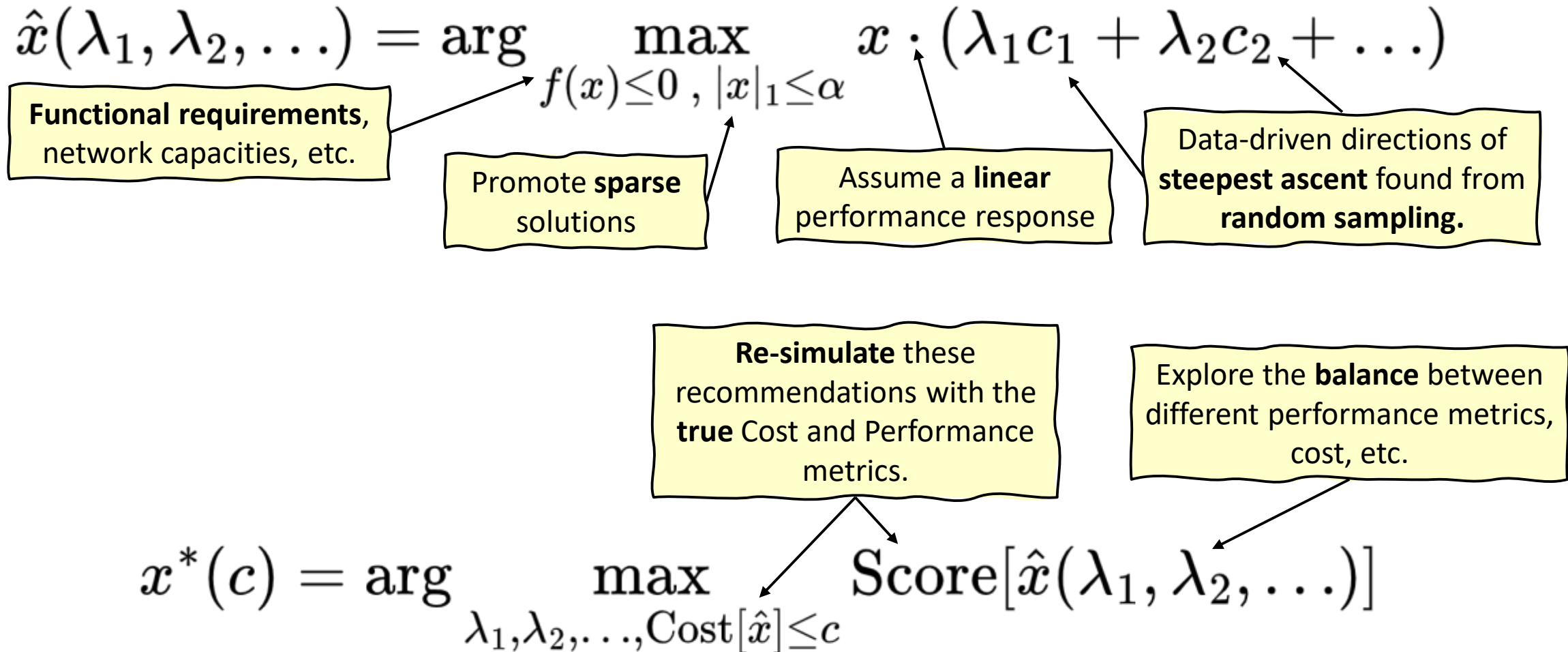
Explore the **balance** between different performance metrics, cost, etc.

$$x^*(c) = \arg \max_{\lambda_1, \lambda_2, \dots, \text{Cost}[\hat{x}] \leq c} \text{Score}[\hat{x}(\lambda_1, \lambda_2, \dots)]$$

# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.



# Linear Models: General Recommendations

Generalizes to the following simple linear recommendation routine:

- **Simulate** the network for random inputs and estimate “**gradient**” **directions**.
- **Guide** a **Linear Program** towards those optimal directions (now with constraints)
- **Evaluate** LP recommendations for **Pareto efficiency**.

$$\hat{x}(\lambda_1, \lambda_2, \dots) = \arg \max_{f(x) \leq 0, |x|_1 \leq \alpha} x \cdot (\lambda_1 c_1 + \lambda_2 c_2 + \dots)$$

Functional requirements,  
network capacities, etc.

Promote **sparse**  
solutions

Assume a **linear**  
performance response

Data-driven directions of  
**steepest ascent** found from  
**random sampling**.

Chose the best-performing  
**recommendation** whose cost is within  
a **variable budget**.

**Re-simulate** these  
recommendations with the  
**true** Cost and Performance  
metrics.

Explore the **balance** between  
different performance metrics,  
cost, etc.

$$x^*(c) = \arg \max_{\lambda_1, \lambda_2, \dots, \text{Cost}[\hat{x}] \leq c} \text{Score}[\hat{x}(\lambda_1, \lambda_2, \dots)]$$



## Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:

## Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.

## Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.
  - **Subroutines:** Deterministic algorithms for sensor/weapon assignment, tracking, etc. running *inside* the simulation.

## Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.
  - **Subroutines:** Deterministic algorithms for sensor/weapon assignment, tracking, etc. running *inside* the simulation.
  - **Superposition:** Targeting networks involve a complex combination of interacting Boolean, Latency, and Tracking behaviors.

# Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.
  - **Subroutines:** Deterministic algorithms for sensor/weapon assignment, tracking, etc. running *inside* the simulation.
  - **Superposition:** Targeting networks involve a complex combination of interacting Boolean, Latency, and Tracking behaviors.

- **Inform** the network recommendation algorithm according to a “**topological prior**”:

 Air Sens    Sea Sens    Both Sens    Both Sens

 Fuse0    Fuse1    Fuse2    Fuse3    Fuse4

 Air Weap    Air Weap    Sea Weap    Sea Weap    Both Weap

# Future Work:

- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.
  - **Subroutines:** Deterministic algorithms for sensor/weapon assignment, tracking, etc. running *inside* the simulation.
  - **Superposition:** Targeting networks involve a complex combination of interacting Boolean, Latency, and Tracking behaviors.

 Air Sens    Sea Sens    Both Sens    Both Sens

 Fuse0    Fuse1    Fuse2    Fuse3    Fuse4

 Air Weap    Air Weap    Sea Weap    Sea Weap    Both Weap

- **Inform** the network recommendation algorithm according to a “**topological prior**”:
  - Use the **connectivity** and **functional relationships** of the network to **motivate** component importance, rather than inferring randomly. (Graph Signal Processing?)

# Future Work:

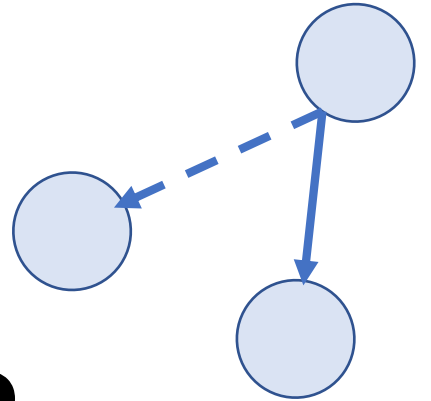
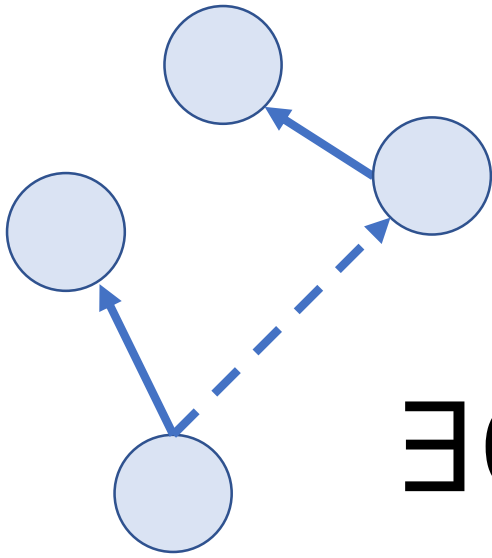
- Tailor network **simulation** capabilities to specific (and more advanced) MBSE needs:
  - **Edge Coloring:** Data classification levels, Shared comms infrastructure, etc.
  - **Subroutines:** Deterministic algorithms for sensor/weapon assignment, tracking, etc. running *inside* the simulation.
  - **Superposition:** Targeting networks involve a complex combination of interacting Boolean, Latency, and Tracking behaviors.

Air Sens    Sea Sens    Both Sens    Both Sens

Fuse0    Fuse1    Fuse2    Fuse3    Fuse4

Air Weap    Air Weap    Sea Weap    Sea Weap    Both Weap

- **Inform** the network recommendation algorithm according to a “**topological prior**”:
  - Use the **connectivity** and **functional relationships** of the network to **motivate** component importance, rather than inferring randomly. (Graph Signal Processing?)
  - Use more advanced **Machine Learning** methods than just Linear Regression! (Graph Neural Networks?)



**$\exists$  Questions  $\cup$  Comments?**

*Thank you to Taylor O. & Michael  
W. for being wonderful mentors  
and teachers!*

